Операционная система и основные команды Linux

Ubuntu Server 24.04 LTS

JDU Cyber Security / 2026 / черновой перевод ChatGPT

Глава 1. Компьютер и операционная система

Вопросы, на которые вы сможете ответить

- Из каких основных компонентов и устройств ввода-вывода состоит компьютер?
- Что общего у персонального компьютера и сервера и чем они различаются?
- Что делает ОС между пользователем, приложениями и оборудованием?
- Чем пространство пользователя отличается от пространства ядра?
- В каком порядке запускается система Linux и её службы?
- Как определяются пользователь, хост и процесс?

1.1 Из чего состоит компьютер

Компьютер — устройство, которое обрабатывает входные данные по инструкциям программы, а результат выводит или сохраняет. Ноутбук, настольный компьютер, смартфон и виртуальный сервер в облаке различаются внешним видом и назначением, но имеют общие основные компоненты.

- CPU (процессор) читает инструкции программы и выполняет вычисления и логические операции.
- Оперативная память временно хранит работающую программу и данные, которые нужны ей сейчас.
- Накопитель сохраняет программы, настройки, документы, данные и журналы после выключения питания. Примеры — SSD и виртуальный диск.
- Устройство ввода принимает действия пользователя или внешние данные: клавиатура, мышь, сенсорный экран.
- Устройство вывода передаёт результат человеку: монитор, динамики, принтер.
- Сетевое устройство обменивается данными с другими компьютерами.

Когда пользователь нажимает клавишу, процессор и накопитель сами по себе не понимают его намерение. Нужен общий механизм, который передаст ввод приложению, выделит ему процессорное время и память, прочитает или запишет файлы и вернёт результат на экран или в сеть. Центральную роль в этом механизме играет операционная система (ОС).

1.2 Персональный компьютер и сервер — компьютеры одного типа

Персональным компьютером (ПК) обычно управляет человек перед ним, используя клавиатуру, мышь и экран. Сервер предоставляет функции или данные другим компьютерам по сети. Веб-сервер отдаёт страницы, сервер базы данных хранит и ищет данные, DNS-сервер сопоставляет имена и IP-адреса.

Сервер не работает по какому-то иному принципу. И ПК, и сервер имеют процессор, память, накопитель, сеть, ОС и приложения. Различаются главным образом назначение, производительность, способ подключения и требования к непрерывной работе. У сервера часто нет постоянно подключённых клавиатуры и монитора; администратор управляет им по сети с другого

компьютера. Инстанс AWS EC2 — виртуальный компьютер в инфраструктуре AWS. В этом курсе мыиспользуем его как сервер.

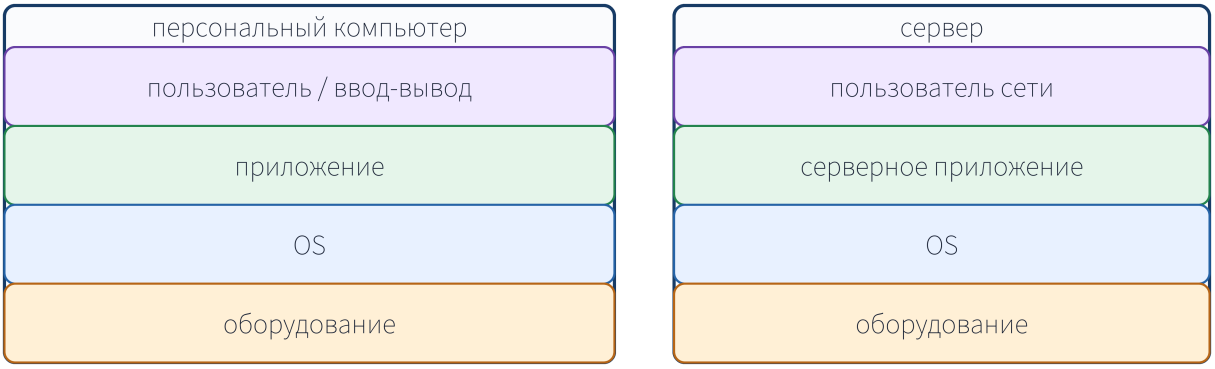


Рисунок 1-1. Несмотря на разное назначение и способы управления, ПК и сервер имеют общую основу: приложения, ОС и оборудование.

1.3 Чем управляет ОС

Когда одновременно работают несколько приложений, нельзя позволять каждому из них бесконтрольно занимать процессор и память или читать чужие файлы. ОС находится между оборудованием и приложениями: распределяет ресурсы, предоставляет общие способы их использования и защищает данные.

В этой книге функции ОС сгруппированы в четыре области.

1. Управление выполнением: запуск программ как процессов, распределение процессорного времени, отслеживание состояния процессов.
2. Управление памятью и файлами: выделение памяти процессам, представление данных накопителя в виде файлов и каталогов.
3. Управление пользователями и защитой: определение того, кто может читать, изменять и запускать объекты, с помощью пользователей, групп и прав доступа.
4. Управление обменом данными: предоставление общих механизмов, например сокетов, для сетевого взаимодействия процессов.

Благодаря общим функциям ОС разработчику приложения не приходится заново писать прямое управление процессором и диском для каждой модели компьютера.

1.4 GUI, CLI и команда

Способ, которым человек передаёт указания компьютеру, называется пользовательским интерфейсом. Два распространённых вида — GUI и CLI.

- GUI (Graphical User Interface, графический интерфейс) показывает окна, значки, меню и кнопки. Команды обычно задаются мышью или касанием.
- CLI (Command Line Interface, интерфейс командной строки) принимает текстовые команды и показывает результат текстом.

Отдельное указание, вводимое в CLI, называется командой. За её именем при необходимости следуют параметры и объект обработки. Например, команда `cat` последовательно читает файл и

выводит его содержимое в стандартный вывод.

```
cat /etc/os-release
```

Здесь `cat` — имя команды, а `/etc/os-release` — путь к читаемому файлу. Введённую строку разбирает командная оболочка, после чего запускается программа `cat`. Она запрашивает у ОС чтение файла; ОС сначала проверяет права доступа и затем передаёт содержимое. Разницу между командой, терминалом и оболочкой, а также основные приёмы работы мы подробно рассмотрим в главе 3.

1.5 Пространство пользователя и пространство ядра

Работающую систему Linux удобно мысленно разделять на пространство пользователя (user space) и пространство ядра (kernel space).

Ядро — центральная часть ОС. Оно управляет процессором, памятью, устройствами, файловой системой, процессами, правами доступа и сетью. Ядро обладает широкими полномочиями и работает в пространстве ядра, отделённом от обычных приложений.

Программы графического интерфейса, терминал, оболочка, команды, веб-сервер и база данных обычно работают в пространстве пользователя. GUI и CLI — не отдельные «пространства»: это способы взаимодействия и соответствующие программы в пространстве пользователя. Приложения тоже работают там и при необходимости обращаются к ядру.

Механизм обращения через эту границу к функциям ядра называется системным вызовом (system call). В этой книге не нужно изучать номера системных вызовов или программирование с ними. Достаточно понимать, что это контролируемый вход к защищённым функциям ядра для программы из пространства пользователя.

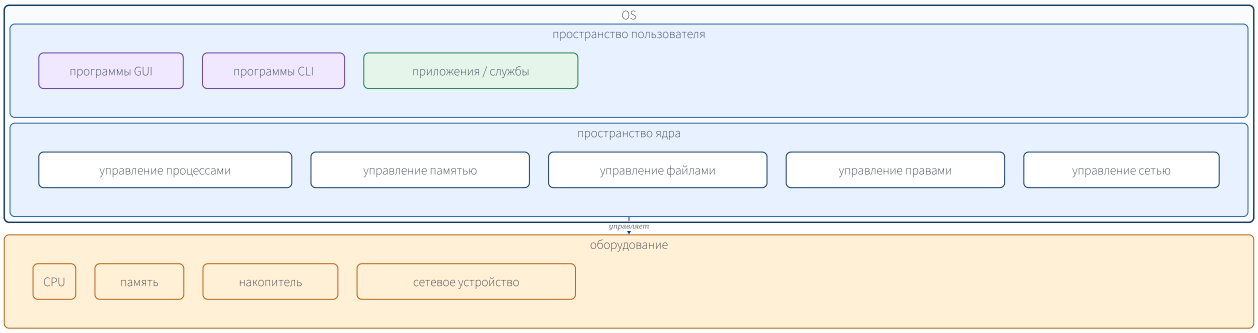


Рисунок 1-2. В широком смысле среда ОС включает программы пространства пользователя и управляющее оборудованием ядро.

Слово «ОС» иногда обозначает преимущественно ядро, а иногда — всю систему вместе с базовыми командами и управляющими программами. Там, где это различие важно, мы отдельно говорим «ядро» и «программы пространства пользователя».

1.6 Linux и Ubuntu

Изначально Linux — название ядра. Но чтобы получить готовую к работе систему, кроме ядра нужны базовые команды, оболочка, библиотеки, управление пакетами и службами. Набор этих компонентов, собранный для установки и обновления, называется дистрибутивом Linux.

Ubuntu, Debian и Fedora — примеры дистрибутивов Linux. Для единой практической среды в этом курсе используется Ubuntu Server 24.04 LTS. Однако мы изучаем не столько конкретный продукт Ubuntu, сколько общие принципы и операции Linux. Историю и различия дистрибутивов рассмотрим в главе 2.

1.7 От включения питания до запуска служб

Программа, лежащая на накопителе, сама по себе ничего не делает. Когда её загружают в память и процессор выполняет её инструкции, возникает процесс. Для различения процессов ОС присваивает каждому PID (идентификатор процесса).

В общих чертах загрузка Linux проходит так:

1. После включения питания прошивка инициализирует процессор, память и накопитель.
2. Загрузчик помещает ядро Linux в память.
3. Ядро подготавливает память, устройства и файловую систему к работе.
4. Ядро запускает первый управляющий процесс в пространстве пользователя.
5. В Ubuntu Server 24.04 таким процессом выступает `systemd`; он получает PID 1.
6. `systemd` читает конфигурацию и зависимости, затем запускает и контролирует службы, необходимые для сети, SSH, журналирования, веб-сервера, базы данных и других функций.

Служба — управляемая программа, которая предоставляет функцию в фоновом режиме без необходимости каждый раз запускать её вручную через экран. При запуске службы начинает работать один или несколько процессов. Например, у сервера базы данных служба служит объектом управления, а её процессы выполняются в памяти, читают и записывают файлы и при необходимости ожидают сетевые запросы.

`systemd` не выполняет сам обработку запросов к базе данных или веб-сайту. Он управляет тем, какие службы, когда, в каком порядке и от имени какого пользователя запускать. PID, родственные связи процессов и сигналы подробнее описаны в главе 8, а `systemd`, службы и юниты — в главе 10.

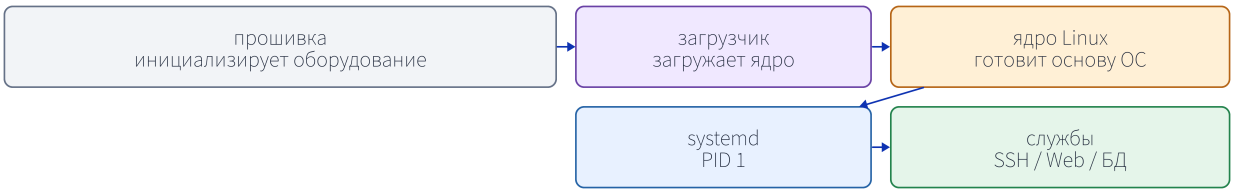


Рисунок 1-3. Ядро запускает первый процесс пространства пользователя; затем systemd запускает процессы служб в нужном порядке.

1.8 Как различать хосты и пользователей

Компьютер или виртуальная машина, подключённые к сети и работающие как самостоятельная среда ОС, называются хостом (host). Для идентификации хосту задают имя хоста (hostname). Это имя компьютера, а не человека.

На одном хосте можно создать несколько учётных записей пользователей. ОС хранит числовые идентификаторы пользователя и его групп. UID (user ID) определяет пользователя, GID (group ID) — группу. Команда `id` показывает имя и UID текущего пользователя, основную и дополнительные

группы.

В практической части используются несколько хостов: ПК студента, AWS CloudShell и Ubuntu EC2. Поля для ввода текста могут выглядеть одинаково, но у каждого хоста собственные ОС, пользователи, процессы и файлы. Чтобы понять, где и от чьего имени вы работаете, проверьте имя пользователя, имя хоста и текущий каталог.

```
id
hostname
pwd
```

`id` показывает сведения о пользователе и группах, `hostname` — имя хоста, `pwd` — текущий рабочий каталог. Управление пользователями объясняется в главе 6, файлы и пути — в главе 4, а связь CloudShell с Ubuntu — в главе 12.

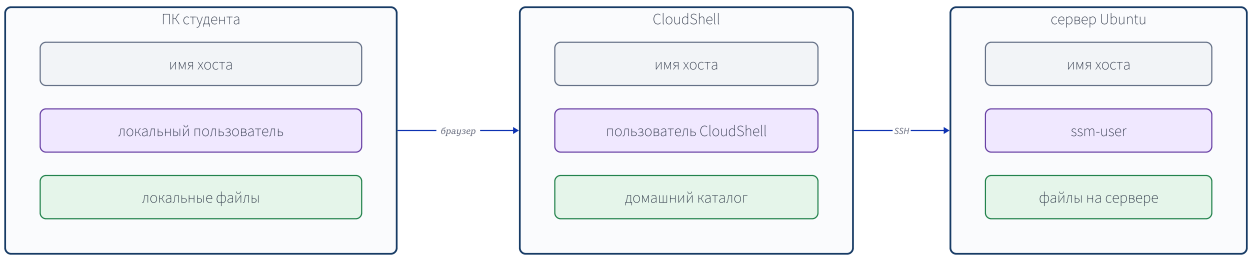


Рисунок 1-4. У каждого хоста собственная среда ОС, в которой отдельно управляются пользователи, процессы и файлы.

Вопросы в конце главы

Вопрос 1. Что общего у ПК и сервера и чем они различаются?

Вопрос 2. Как связаны GUI, CLI, пространство пользователя и пространство ядра?

Вопрос 3. Чем отличаются программа, процесс и служба?

Вопрос 4. Что делает systemd с PID 1?

Вопрос 5. Чем имя хоста отличается от имени пользователя?

Ответы на вопросы главы

Вопрос 1

Ответ: оба являются компьютерами с процессором, памятью, накопителем, сетью, ОС и приложениями. ПК преимущественно рассчитан на непосредственное взаимодействие с человеком, а сервер — на предоставление функций и данных другим компьютерам по сети.

Вопрос 2

Ответ: GUI и CLI — способы управления; реализующие их программы обычно работают в пространстве пользователя. Через системные вызовы они используют функции пространства ядра. Ядро управляет оборудованием и ресурсами ОС.

Вопрос 3

Ответ: программа — набор инструкций, сохранённых на накопителе. Процесс — экземпляр выполняемой программы, загруженный в память. Служба — управляемая `systemd` или другой системой единица для предоставления постоянной функции; фактически её выполняют один или несколько процессов.

Вопрос 4

Ответ: в Ubuntu Server 24.04 первым управляющим процессом пространства пользователя, который запускает ядро, является `systemd` с PID 1. По конфигурации и зависимостям он запускает, останавливает и отслеживает службы.

Вопрос 5

Ответ: имя хоста определяет компьютер или виртуальную машину с ОС, а имя пользователя — учётную запись с определёнными правами на этом хосте. На одном хосте может быть несколько пользователей.

Источники

- Linux Kernel Organization, The Linux Kernel documentation (проверено 2026-09-21).
- Linux man-pages, intro(2) — introduction to system calls (проверено 2026-09-21).
- Ubuntu Manpages, boot(7) — system bootup process (проверено 2026-09-21).
- Ubuntu Server documentation, Ubuntu Server documentation (проверено 2026-09-21).

Глава 2. От Unix до современных ОС

Зачем изучать историю

Цель исторического обзора — не запомнить даты. Он помогает понять, почему Linux, macOS и Windows используют похожие термины, но нередко разные команды. Важно избежать ошибок: «Linux прямо унаследовал исходный код Unix», «macOS — разновидность Linux», «Windows произошла от Unix».

2.1 Идеи Unix

Разработка Unix началась в Bell Labs в 1969 году. По мере развития Unix распространил принципы, применяемые и в современных ОС: иерархическую файловую систему, процессы, работу нескольких пользователей и сочетание небольших инструментов.



Фото 2-1. Сохранившийся DEC PDP-7 — модель, использованная при ранней разработке Unix. Снимок сделан в музее США в 2018 году: ComputerGeek7066, Wikimedia Commons, CC BY-SA 4.0; изображение не изменено.

На фотографии не конкретная машина из Bell Labs. PDP-7 выпускался в 1960-е годы, и на компьютере этой модели работал ранний Unix.

Эти идеи связаны с нашей практикой: файлы расположены в иерархии каталогов; работающие программы наблюдаются как процессы; права разграничиваются по пользователям и группам; вывод `grep` передаётся другой команде или в файл. Такая модель работы давно применяется в Unix-подобных системах.

«Unix» — не название единого набора команд. Системы, испытавшие его влияние, не обязательно используют один и тот же исходный код или официально сертифицированы на торговую марку UNIX. Систему с похожей организацией и интерфейсами обычно называют Unix-подобной

(Unix-like). Классификацию по принципам устройства следует отличать от формальной сертификации продукта.

2.2 GNU и свободная Unix-совместимая среда

Проект GNU был объявлен в 1983 году. Его цель — создать свободную программную систему, совместимую с Unix. Были разработаны оболочки, компиляторы и многие базовые утилиты. Во многих современных дистрибутивах Linux вокруг ядра работает разнообразное ПО, в том числе инструменты GNU.

Командная оболочка (shell) — программа пространства пользователя. Она читает введённый текст, разбирает его как команду и запускает нужные программы. Оболочка не ограничивается запуском одной команды: она разбирает аргументы, организует перенаправление ввода и вывода и конвейеры, подставляет значения переменных среды, последовательно выполняет команды сценария. Так она связывает действия пользователя с множеством инструментов.

Bash — одна из оболочек проекта GNU. Название происходит от «Bourne-Again Shell». Bash не является ни ядром Linux, ни окном терминала. Он разбирает текст, введённый в терминале Ubuntu, и запускает программы, например `ls` и `cp`. Операция `cd`, меняющая состояние самой оболочки, выполняется встроенной командой Bash. Отличия терминала, оболочки и команды подробно рассмотрены в главе 3.

Само по себе ядро ещё не составляет систему, с которой работает студент. Bash предоставляет оболочку, GNU coreutils — базовые команды вроде `ls` и `cp`. Практически полезная система возникает при соединении ядра с менеджером пакетов, `systemd`, библиотеками, файлами конфигурации и приложениями.

2.3 Linux — независимо созданное ядро

Разработка ядра Linux началась в 1991 году. Оно создавалось с функциями и интерфейсами, похожими на Unix, но не возникло как прямая ветвь исторического исходного кода Unix. Поэтому на схеме происхождения ОС связь Unix и Linux следует изображать как влияние идей или стремление к совместимости, а не как прямое наследование кода.

Ядро Linux отвечает за низкоуровневую работу процессов, памяти, файловой системы, устройств и сети. Ubuntu — дистрибутив, который объединяет ядро Linux с пользовательскими инструментами, пакетами, начальными настройками и политикой выпусков.

Семейство Windows NT, напротив, проектировалось отдельно от Unix-подобных систем. `cmd.exe` и PowerShell в Windows тоже являются командными оболочками, но отличаются от Bash синтаксисом, встроенными командами, записью путей и способами администрирования. Существуют похожие названия команд и программы, работающие на нескольких ОС, однако рассчитывать на полную взаимозаменяемость команд Unix-подобных систем и Windows нельзя. Bash — не сам Linux, а одна из оболочек, доступных и в Linux, и в macOS.

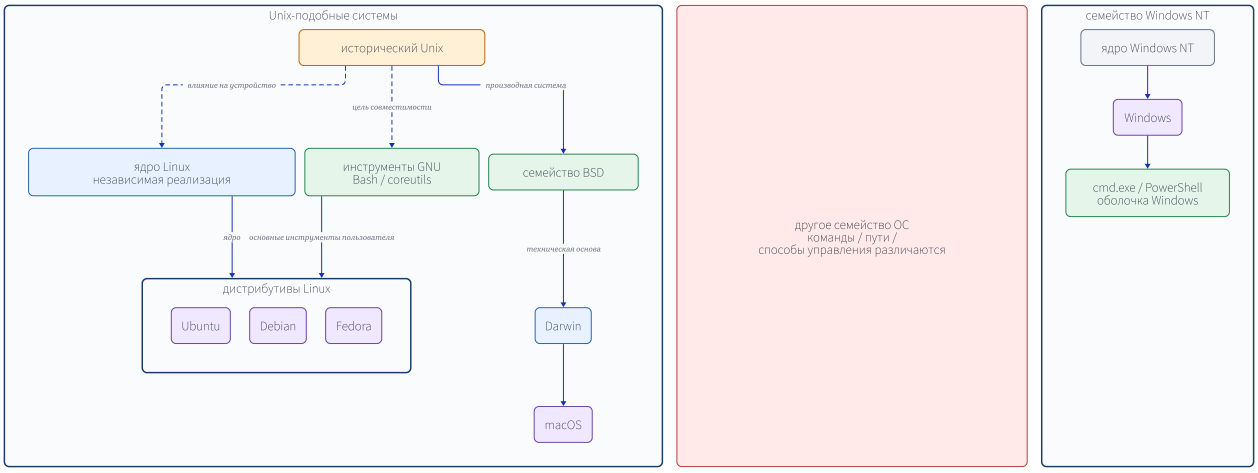


Рисунок 2-1. Unix-подобное семейство и семейство Windows NT имеют разное происхождение; одинаковые команды не гарантированы. Linux независимо реализует похожие на Unix интерфейсы, а macOS не происходит от Linux.

2.4 Дистрибутив выбирает и объединяет компоненты

На ядре Linux построены разные дистрибутивы: Ubuntu, Debian, Fedora и другие. Каждый выбирает программное обеспечение, формат пакетов, правила обновления, начальные настройки и срок поддержки.

Для практики в этом курсе используется Ubuntu Server 24.04 LTS. Команды `ls` и `grep` часто работают и в других дистрибутивах, но команды управления пакетами, расположение настроек, механизмы безопасности и начальная конфигурация служб могут различаться. Инструкции для Red Hat/RHEL нельзя без проверки применять к Ubuntu.

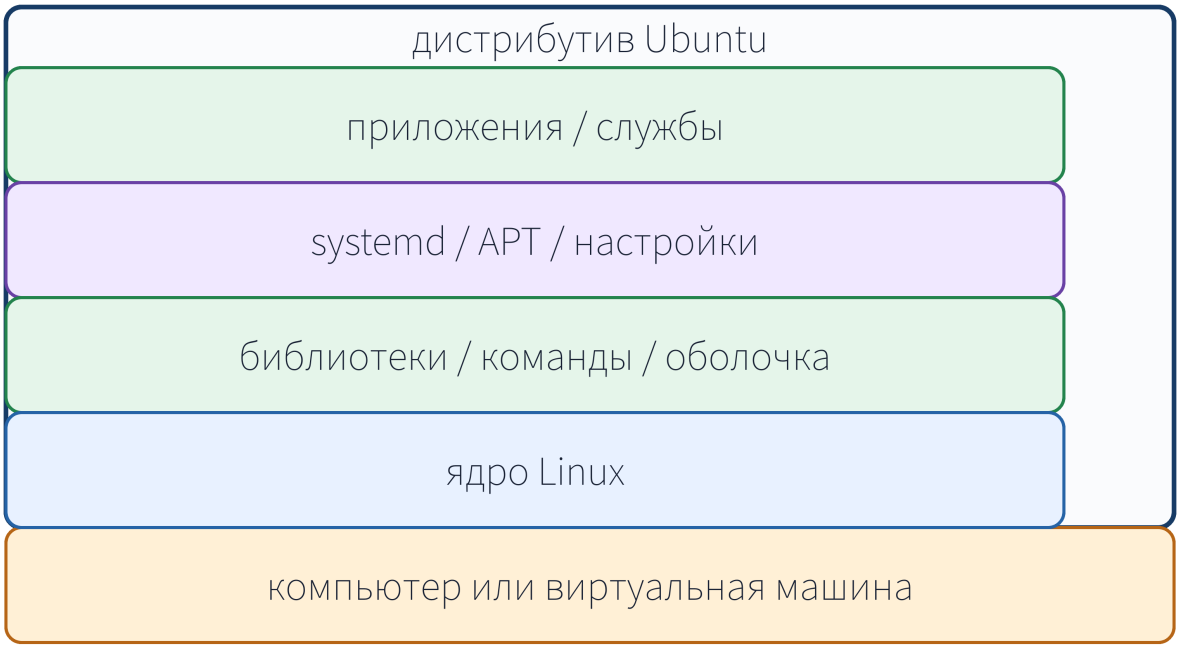


Рисунок 2-2. Дистрибутив объединяет ядро Linux, библиотеки, базовые команды, системное управление, пакеты и приложения. Ubuntu и Linux не следует считать полными синонимами.

2.5 Основа macOS — Darwin

В основе современной macOS лежит Darwin. По документации Apple он включает технологии Mach, BSD и собственные разработки Apple. Благодаря технологиям BSD, связанным с процессами, правами доступа и сетью, в терминале macOS доступны команды и пути, похожие на Unix-подобные.

Однако macOS — не дистрибутив Linux: её ядро не Linux. Кроме того, macOS включает графическую систему и платформенные компоненты Apple, не сводящиеся к Darwin. Следует также различать старую Classic Mac OS и линию на основе Darwin, начавшуюся с Mac OS X.

2.6 Windows — отдельная линия развития

У ранних потребительских версий Windows существовала линия на основе MS-DOS, например Windows 95/98. Линия Windows NT, ведущая к современной Windows, была спроектирована отдельно. Windows 2000, XP и более поздние версии нельзя изображать как простое ответвление Unix.

В Windows есть режим ядра и режим пользователя, процессы, права доступа к файлам, службы, сетевые сокеты и CLI. Эти общие черты объясняются тем, что современные ОС решают похожие задачи, а не происхождением Windows от Unix. Наличие PowerShell или Windows Subsystem for Linux (WSL) также не превращает само ядро Windows в Linux.

2.7 Как сопоставлять общее и различное

Критерий	Ubuntu/Linux	macOS	Windows
Ядро	Linux	Darwin (включая Mach и BSD)	семейство Windows NT
Типичный CLI	оболочка и Unix-подобные инструменты	оболочка и инструменты BSD/Unix	PowerShell, cmd и др.
Пример пути	/home/user	/Users/user	C:\Users\user
Установка ПО	менеджер пакетов дистрибутива	App Store, установщики, менеджеры пакетов	Microsoft Store, установщики, менеджеры пакетов
Роль в курсе	практическая среда	объект сравнения	объект сравнения

Даже при одинаковом назначении конкретные команды, настройки по умолчанию и детали прав доступа различаются между ОС. Изучая общие принципы на Ubuntu, учитесь затем искать не «ту же команду», а механизм, выполняющий ту же задачу в другой системе.

Вопросы в конце главы

Вопрос 1. Объясните разницу между Linux и Ubuntu с помощью терминов «ядро» и «дистрибутив».

Вопрос 2. Что такое командная оболочка? Приведите пример Bash.

Вопрос 3. Почему macOS нельзя назвать дистрибутивом Linux?

Вопрос 4. Доказывает ли наличие процессов и прав доступа в Windows её прямое происхождение от Unix? Почему?

Ответы на вопросы главы

Вопрос 1

Ответ: Linux — ядро, управляющее процессами, памятью, устройствами и другими ресурсами. Ubuntu — дистрибутив Linux, в котором ядро объединено с инструментами GNU, оболочкой, библиотеками, управлением пакетами и настройками.

Вопрос 2

Ответ: оболочка разбирает введённые пользователем команды и выполняет запуск программ, передачу аргументов, перенаправления, конвейеры, подстановку переменных и сценарии. Bash — оболочка проекта GNU, а не само ядро Linux.

Вопрос 3

Ответ: основой macOS является Darwin с технологиями Mach и BSD. Поскольку её ядро не Linux, macOS не является дистрибутивом Linux.

Вопрос 4

Ответ: нет. Процессы и права доступа нужны многим современным ОС. Windows NT развивалась отдельно от Unix; `cmd.exe` и PowerShell отличаются от среды Bash синтаксисом, командами, путями и администрированием.

Источники

- Dennis M. Ritchie, The Evolution of the Unix Time-sharing System (проверено 2026-09-18).
- Bell Labs, A history of computing at Bell Research Laboratories (1937-1975) (проверено 2026-09-24; ранний Unix и PDP-7).
- Wikimedia Commons, DEC PDP-7.jpg (проверено 2026-09-24; ComputerGeek7066, CC BY-SA 4.0; музейная машина в 2018 году).
- GNU Project, Initial Announcement (проверено 2026-09-18).
- GNU Project, What is a shell? (проверено 2026-09-21).
- GNU Project, What is Bash? (проверено 2026-09-21).
- Linux Kernel Documentation, Introduction (проверено 2026-09-18).
- Apple, Kernel Architecture Overview (проверено 2026-09-18; архивная страница используется только для истории и устройства).
- Microsoft, Windows NT Build History (проверено 2026-09-18; исторический источник).
- Microsoft Learn, Windows commands (проверено 2026-09-21).

Глава 3. GUI, CLI, терминал и командная оболочка

3.1 Отделяем интерфейс от исполнителя

GUI (графический пользовательский интерфейс) позволяет управлять компьютером визуально: через окна, значки, меню и указатель. В CLI (интерфейсе командной строки) вводят текстовую команду и получают текстовый результат. Оба способа служат входом для указаний человека. GUI не обязан «превращаться в CLI», чтобы работать.

Терминал (terminal) — окно или среда соединения для ввода текста и показа результата. Командная оболочка (shell) — программа, которая читает введённый текст, разбирает команды и выполняет их. В практической среде в основном используется Bash.

```
Человек → терминал → Bash (оболочка) → процесс команды → функции ядра
```

Терминал и оболочка — разные вещи. Терминал обеспечивает обмен символами; оболочка определяет смысл введённого текста и организует выполнение.

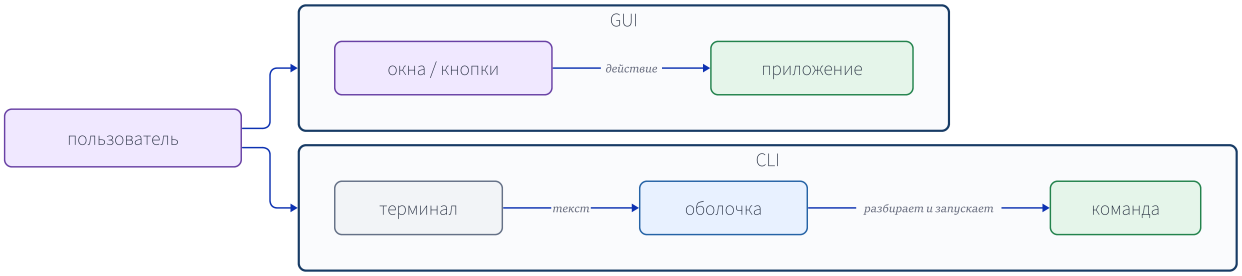


Рисунок 3-1. GUI и CLI — два способа взаимодействия; при использовании CLI важно различать терминал и оболочку.

3.2 Приглашение показывает место ввода

Строка, которую оболочка выводит, ожидая ввода, называется приглашением командной строки (prompt). В ней часто есть имя пользователя, имя хоста и текущий каталог, но содержание зависит от настроек.

Символ `$` в следующем примере изображает приглашение; самостоятельно вводить его не нужно.

```
$ pwd
/home/ssm-user
```

Здесь вводится `pwd`, а следующая строка — пример вывода. В этой книге `$` иногда используется для различения вводимой команды и результата.

3.3 Команда, аргумент и параметр

Общую форму читают так:

```
команда  параметр  аргумент
ls       -l        /srv
```

`ls` — выполняемая команда, `-l` — параметр подробного вывода, `/srv` — аргумент, указывающий объект. Во многих командах короткие параметры объединяются, например `ls -la`. Но одинаковая буква не обязательно означает одно и то же: `grep -n` добавляет номера строк, а `tail -n 5` задаёт количество выводимых строк.

Пробелы разделяют части командной строки. Чтобы передать строку с пробелами как один аргумент, заключайте её в кавычки.

```
printf '%s\n' 'JDU SSH transfer test'
```

Кавычки, подстановка переменных и перенаправление подробнее рассматриваются в главе 5.

3.4 Встроенные команды оболочки и внешние программы

Команды, вводимые в оболочке, в основном делятся на встроенные команды и внешние программы.

Встроенная команда (shell builtin) реализована самой Bash; отдельный исполняемый файл искать и запускать не нужно. Например, `cd` меняет рабочий каталог текущей оболочки Bash и поэтому является встроенной командой.

Для внешней команды оболочка ищет программу на накопителе и запускает её. `ls`, `mkdir`, `grep` обычно доступны сразу после установки Ubuntu, но это внешние программы, а не встроенные функции Bash. Следовательно, «доступна по умолчанию» и «встроена в оболочку» — разные характеристики.

Часть внешних программ устанавливается вместе с ОС, другие добавляются при необходимости. `python3` — внешняя команда для запуска Python. Она доступна, только если Python установлен. В некоторых конфигурациях Ubuntu он уже есть, в других его устанавливают пакетом. Установка пакетов рассматривается в главе 9.

```
type cd
type mkdir
type python3
```

Команда `type` показывает, как Bash воспринимает введённое имя: как встроенную команду или как внешнюю программу с определённым расположением файла. Если имя не найдено, такая команда в текущей среде недоступна.

Иногда `type` сообщает об алиасе (alias). Алиас — короткое альтернативное имя для более длинной команды или часто используемого набора параметров. Например, в некоторых средах `ll` обозначает `ls -alF`. Наличие и значение алиасов зависят от настроек.

Для поиска внешней команды оболочка по очереди проверяет каталоги, перечисленные в настройке `PATH`. Заранее запоминать расположение каждого исполняемого файла не нужно. Сначала используйте `type`, чтобы узнать, считается ли имя встроенной командой, внешней программой или алиасом.

3.5 Как оболочка выполняет команду

Оболочка не передаёт введённый текст ядру без обработки. В общих чертах она:

1. Разбирает строку на имя команды, параметры и аргументы.

2. По правилам обрабатывает кавычки, переменные среды, подстановку команд `$(...)` и шаблоны имён файлов.
3. Выполняет встроенную команду сама либо ищет исполняемый файл внешней программы.
4. Подготавливает потоки ввода и вывода, перенаправления и конвейеры.
5. Запускает процесс и ждёт его завершения на переднем плане либо оставляет работать в фоне.
6. Получает код завершения процесса.

Это помогает понять, на каком этапе возникла проблема. Если команда не найдена, проверьте её имя и поиск внешней программы. Если обработан не тот объект, проверьте параметры и аргументы. Ввод-вывод и процессы подробнее описаны в главах 5 и 8.

3.6 Полезные действия и команды для ориентации

Если в CLI вы не знаете, где находитесь или что означает команда, остановитесь и уточните ситуацию.

Действие или команда	Что показывает или делает
Ctrl+C	Просит остановить текущую операцию и вернуть приглашение.
pwd	Показывает текущий рабочий каталог.
cd КАТАЛОГ	Переходит в другой рабочий каталог.
ls	Показывает файлы и каталоги текущего каталога.
grep 'ТЕКСТ' ФАЙЛ	Ищет в файле строки с указанным текстом.
type ИМЯ_КОМАНДЫ	Показывает, является имя встроенной командой, внешней программой или алиасом.
help ИМЯ_КОМАНДЫ	Показывает справку о встроенной команде Bash.
exit	Завершает текущую оболочку и возвращает к предыдущему состоянию.

`pwd`, `cd` и `ls` помогают понять, «где и с чем я работаю». `grep` ищет нужные строки в тексте и журналах. `type` и `help` помогают выяснить тип команды и правила её использования. `grep` и `ls` — полезные внешние программы, а не встроенные команды оболочки.

3.7 Справочные руководства

Для внешних команд отправной точкой служит `man`, для встроенных команд Bash — `help`.

```
man ls
help cd
grep --help
```

В руководстве `NAME` кратко описывает назначение команды, `SYNOPSIS` — форму записи, `DESCRIPTION` — поведение, `OPTIONS` — параметры. Квадратные скобки `[ ]` в записи синтаксиса обычно означают необязательную часть; сами скобки вводить не нужно. Просмотр `man` завершается клавишей `q`.

3.8 Успех операции и код завершения

Помимо текста на экране, команда возвращает оболочке небольшое целое число — код завершения (exit status). В Unix-подобных системах `0` означает успешное завершение, а ненулевое значение — неуспех или ошибку.

```
test -e /etc/os-release
echo $?
```

Специальная переменная `$?` содержит код завершения последней команды. Следующая команда сразу заменит это значение, поэтому проверять его нужно непосредственно после интересующей операции. Ненулевые значения не всегда указывают на одну причину. Например, `grep` возвращает `1`, если совпадений нет, и `2` при ошибке выполнения, например при отсутствии файла. Важно учитывать не только сообщение на экране, но и то, что сама программа считает успехом или неуспехом.

Вопросы в конце главы

Вопрос 1. Чем терминал отличается от оболочки?

Вопрос 2. Почему `cd` должна быть встроенной командой оболочки?

Вопрос 3. Почему «команда доступна сразу» не означает «команда встроена в оболочку»?

Вопрос 4. Как определить тип команды?

Ответы на вопросы главы

Вопрос 1

Ответ: терминал — место или среда соединения для ввода и вывода текста. Оболочка — программа, которая разбирает введённый текст, выполняет встроенные действия и запускает внешние программы.

Вопрос 2

Ответ: `cd` должна менять рабочий каталог самой текущей оболочки. Если каталог изменит только отдельный внешний процесс, текущий каталог исходной оболочки останется прежним.

Вопрос 3

Ответ: по умолчанию могут быть установлены и внешние программы. Например, `mkdir` часто доступна сразу в Ubuntu, но не встроена в Bash, тогда как `cd` встроена.

Вопрос 4

Ответ: выполните `type ИМЯ_КОМАНДЫ`. Результат покажет встроенную команду, внешнюю программу или алиас.

Источники

- Ubuntu Server documentation, Welcome to the terminal (проверено 2026-09-18; актуальную структуру страницы следует перепроверить при публикации).
- GNU Project, Bash Reference Manual (получение страницы 2026-09-18 завершилось по тайм-ауту; содержание также сверяется с `help` и `man bash` в Ubuntu 24.04).

Глава 4. Файлы, каталоги, пути и редактирование

4.1 У файла есть содержимое и метаданные

Помимо данных файла ОС хранит его метаданные (metadata): имя, тип, владельца, группу, права доступа, размер, время последнего изменения и другое. Два файла с одинаковым выводом `cat` могут считаться разными по состоянию в ОС, если у них разные владельцы или права.

```
ls -l /srv/jdu-share/README.txt
stat /srv/jdu-share/README.txt
```

`ls -l` выводит основные сведения о содержимом каталога списком. `stat` подробно показывает метаданные одного указанного объекта.

4.2 Каталог связывает имя с объектом

Представление каталога только как «коробки, куда физически складывают файлы» неточно. Каталог — особый файл, связывающий имена файлов и других каталогов с соответствующими объектами и образующий иерархию. Путь `/srv/jdu-share/README.txt` задаёт движение от корневого каталога `/` через имена `srv`, `jdu-share` и `README.txt`.

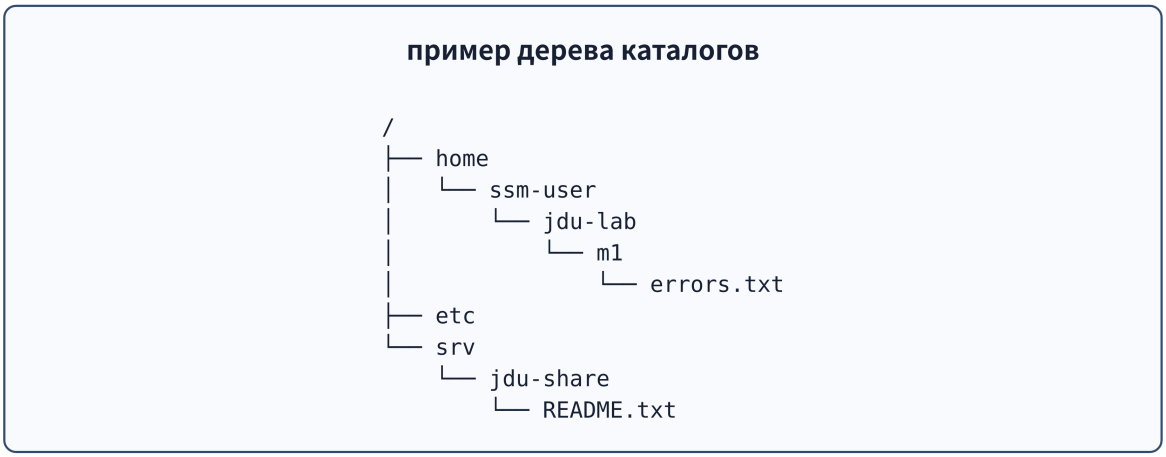


Рисунок 4-1. Иерархия в виде дерева, похожая на вывод `tree`: положение файла определяется последовательностью ветвей.

В Linux вся система представлена одним деревом каталогов с вершиной `/`. Даже физически разные диски и виртуальные файловые системы можно присоединить к определённому каталогу и представить частью общего дерева. Такое присоединение называется монтированием (mount). Здесь достаточно понять принцип; форматирование диска и практическое монтирование изучаются в последующем разделе о хранении данных.

Корневой каталог `/` — начало всего дерева. Домашний каталог — место для файлов конкретного пользователя. Например, домашний каталог `ssm-user` — `/home/ssm-user`: от `/` нужно пройти через `home` и `ssm-user`. `/home` — родительский каталог для домашних каталогов разных пользователей, а не домашний каталог самого `ssm-user`. `/` и `/home/ssm-user` имеют разные роли и положение. Похожее по названию `/root` — ещё одно отдельное место, не корневой каталог `/`.

4.3 Абсолютные и относительные пути

Путь, начинающийся с `/`, — абсолютный путь. Он обозначает одно и то же место независимо от текущего каталога.

```
/srv/jdu-share/README.txt
```

Путь без начального `/` — относительный путь. Его отсчитывают от текущего рабочего каталога.

```
cd /srv/jdu-share
cat README.txt
```

`pwd` показывает текущий каталог. `.` означает «этот каталог», `..` — «родительский каталог». `~` (тильда) разворачивается оболочкой в домашний каталог текущего пользователя.

```
cd ~/jdu-lab/m1
pwd
cd ..
pwd
```

`~` не означает корневой каталог `/`. Его значение меняется с пользователем и хостом. Например, у `ssm-user` это может быть `/home/ssm-user`, а после `su - jduops` — домашний каталог `jduops`, например `/home/jduops`. На другом хосте оно относится к текущему пользователю этого хоста. Текущий каталог показывает `pwd`, а путь домашнего каталога — `printf '%s\n' "$HOME"`. `HOME` — переменная среды; переменные объясняются в главе 5.

4.4 Часто встречающиеся каталоги Ubuntu

Путь	Назначение в этой книге
/home	Домашние каталоги обычных пользователей.
/etc	Общесистемные настройки и идентификационные файлы служб; здесь же находится /etc/os-release.
/usr/bin	Исполняемые файлы многих обычных команд.
/var/log	Файлы журналов системы и служб; не все журналы находятся только здесь.
/srv	Данные, предоставляемые системой или службой; в практике — общие каталоги и содержимое веб-сервера.
/tmp	Временные файлы; доступен для записи многим пользователям, но не подходит для постоянного хранения.
/proc	Виртуальная файловая система, через которую ядро представляет состояние процессов и системы как файлы.

Эта таблица не означает, что каждый объект обязан находиться строго в указанном месте. Детали зависят от политики дистрибутива и проектирования приложения.

4.5 Создание, копирование, перемещение и удаление

```
mkdir -p ~/jdu-lab/p1/practice01/config
cp inbox/config/training.conf practice01/config/training.conf
mv old-name.txt new-name.txt
rm unwanted.tmp
```

- `mkdir -p` создаёт и отсутствующие родительские каталоги; существующий целевой каталог не вызывает ошибку.
- `cp SOURCE DESTINATION` копирует объект. Не перепутайте порядок источника и назначения.
- `mv` переносит файл или меняет его имя.
- `rm` удаляет файл сразу, не помещая его в привычную по GUI «корзину». Перед выполнением проверьте путь через `pwd` и `ls`.

В упражнениях для начинающих не используется опасная команда `rm -rf`, способная без предупреждения удалить всё дерево каталога. Даже перед удалением ненужных файлов `.tmp` сначала выведите их список через `find`.

```
find ~/jdu-lab/p1/practice01 -type f -name '*.tmp' -print
```

Одинарные кавычки вокруг `'*.tmp'` запрещают оболочке преждевременно разворачивать шаблон и передают его самой команде `find`. Визуально проверьте найденные файлы, а затем удаляйте их по одному либо действуйте только в безопасных границах, указанных в упражнении.

4.6 Другие команды проверки

```
ls -la
ls -ld /srv/jdu-share
tree ~/jdu-lab/m1/case01
stat -c '%U:%G %a %n' /srv/jdu-share/README.txt
```

- `ls -la` подробно перечисляет все объекты текущего каталога, включая скрытые имена с начальной точкой.
- `ls -ld DIR` показывает метаданные самого каталога `DIR`, а не список его содержимого.
- `tree` наглядно показывает дерево каталогов. В нашей практической среде она установлена заранее.
- `stat -c` выводит выбранные поля в заданном формате: `%U` — имя владельца, `%G` — имя группы, `%a` — права в восьмеричной записи, `%n` — имя или путь.

Выбирайте команду в зависимости от вопроса: дерево (`tree`), список содержимого (`ls`) или подробные свойства объекта (`stat`). Ни одна из этих команд не заменяет остальные во всех случаях.

4.7 Редактирование в nano

`nano FILE` — простой текстовый редактор внутри терминала. Перед редактированием перейдите в нужный каталог и проверьте место и целевой файл через `pwd` и `ls`.

```
cd ~/jdu-lab/m0
pwd
nano observation.env
```

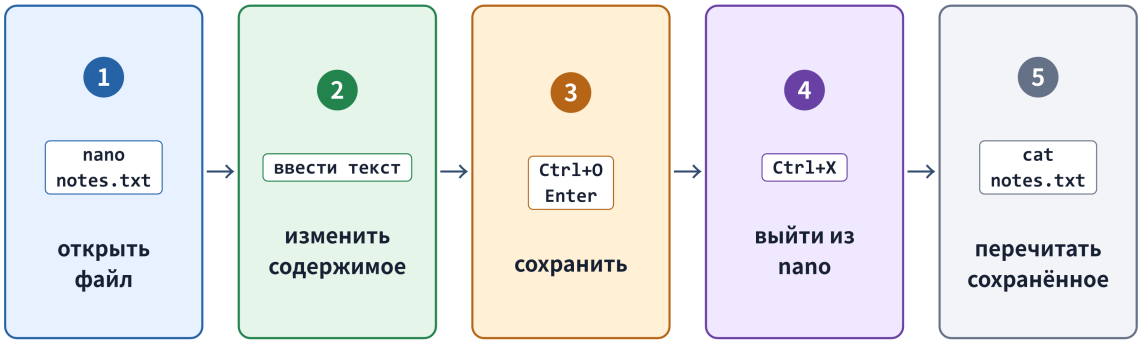


Рисунок 4-2. Основные действия в nano; знак ^ внизу экрана обозначает клавишу Ctrl.

1. Перемещайте курсор стрелками и редактируйте текст обычным вводом.
2. Нажмите `Ctrl+O`, чтобы начать запись файла.
3. Когда внизу появится имя файла, проверьте путь и подтвердите сохранение клавишей `Enter`.
4. Нажмите `Ctrl+X`, чтобы выйти из nano.
5. Снова прочитайте результат, например `cat observation.env`, и убедитесь, что изменения сохранились.

При выходе с несохранёнными изменениями nano спрашивает, сохранить ли их. В англоязычном интерфейсе `Y` (Yes) сохраняет, `N` (No) отбрасывает, `Ctrl+C` отменяет действие. Не нажимайте клавиши наугад, не прочитав сообщение.

4.8 Важны и права на родительские каталоги

Чтобы прочитать файл, ОС проходит путь по порядку от начала. На каждом промежуточном каталоге она проверяет, разрешён ли текущему пользователю проход. Если хотя бы один каталог пройти нельзя, файл не удастся прочитать даже при наличии права чтения `r` у самого файла. `namei -l` показывает все компоненты пути по отдельности вместе с их владельцами и правами.

```
namei -l /srv/jdu-web/index.txt
```

Для каталога право выполнения `x` означает не «запуск файла-программы», а возможность пройти через каталог и обратиться к именам внутри него. Подробности разбираются в главе 7.

Вопросы в конце главы

Вопрос 1. Чем корневой каталог отличается от домашнего?

Вопрос 2. Что показывают `ls -l DIR` и `ls -ld DIR`?

Вопрос 3. Что следует проверить через `namei -l`, если файл с правом чтения `r` не открывается?

Ответы на вопросы главы

Вопрос 1

Ответ: корневой каталог `/` — начало всего дерева каталогов. Домашний каталог — личное рабочее место пользователя, например `/home/ssm-user`. `~` обозначает домашний каталог текущего пользователя.

Вопрос 2

Ответ: `ls -l DIR` подробно показывает объекты внутри `DIR`, а `ls -ld DIR` — свойства самого каталога `DIR`.

Вопрос 3

Ответ: проверьте у каждого родительского каталога, начиная с начала пути, есть ли у текущего пользователя право прохода `x`. Одного непроходимого каталога достаточно, чтобы нельзя было прочитать файл даже с `r`.

Источники

- Linux man-pages, path_resolution(7) (проверено 2026-09-18).
- Руководства `man hier`, `man find`, `man stat` и интерфейс nano в Ubuntu 24.04 (проверяются в рабочей системе при подготовке).

Глава 5. Работа с текстом и потоками ввода-вывода

5.1 Почему настройки и журналы часто текстовые

В Ubuntu и других системах Linux многие настройки и журналы хранятся как обычный текст. Его удобно читать, сравнивать и искать без специальной графической программы, а также передавать другим командам для обработки. Но не все настройки и журналы — простые текстовые файлы. Например, системный журнал systemd хранится в двоичном формате и просматривается специальной командой.

5.2 Стандартный ввод, вывод и поток ошибок

У каждого процесса есть три базовых потока данных: стандартный ввод (stdin), стандартный вывод (stdout) и стандартный поток ошибок (stderr).

- stdin принимает обычные входные данные, например с клавиатуры или из вывода предыдущей команды (файловый дескриптор 0).
- stdout передаёт обычный результат на экран терминала, в следующую команду или в файл (дескриптор 1).
- stderr передаёт ошибки и предупреждения отдельно от обычного результата (дескриптор 2).

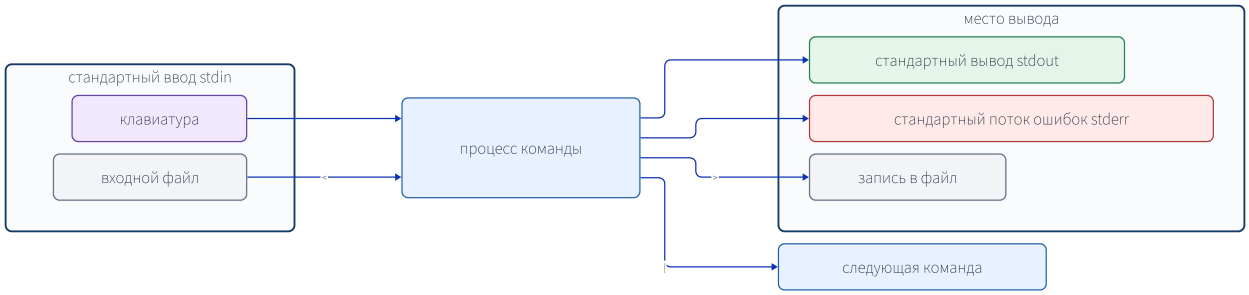


Рисунок 5-1. Три стандартных потока программы. `>` обычно направляет stdout в файл, а `|` соединяет stdout команды слева со stdin команды справа.

Если выполнить `cat incident.log`, файл станет источником данных, а экран терминала — местом показа stdout. Не сам файл «перелетает» в терминал: процесс `cat` читает его содержимое и записывает прочитанный текст в stdout.

5.3 `grep` выбирает нужные строки

```
grep 'ERROR' incident.log
```

`grep` просматривает входные данные построчно и выводит в stdout целые строки, совпавшие с заданным шаблоном.

```
grep -F 'ERROR' incident.log
```

Параметр `-F` рассматривает шаблон как буквальную строку, а не регулярное выражение. Для простого слова `ERROR` результат часто совпадает, но `-F` полезен, когда нужно найти специальные символы без экранирования.

`grep -n` добавляет перед найденной строкой её номер. Если в другой файл нужны только исходные строки, не добавляйте `-n`.

Важен и код завершения: обычно `0` означает найденное совпадение, `1` — отсутствие совпадений, `2` — ошибку выполнения, например отсутствующий файл. Когда вывод пуст, код помогает отличить «строки нет» от «ошиблись в имени файла».

5.4 `tail` читает конец файла

```
tail -n 5 incident.log
```

`tail -n 5` выводит последние пять строк в исходном порядке. Здесь `-n` задаёт число строк, а у `grep -n` добавляет номера строк: значение одной буквы зависит от команды. Проверяйте параметры по руководству, а не запоминайте их без контекста.

`tail -f` непрерывно показывает новые строки, добавляемые в файл. Остановить наблюдение можно через `Ctrl+C`.

5.5 Сохранение результата перенаправлением

```
grep 'ERROR' incident.log > errors.txt
tail -n 5 incident.log > recent.txt
```

`>` — механизм оболочки, переключающий stdout команды на файл. Если файл справа уже существует, оболочка до запуска команды обнуляет его размер, после чего запись идёт поверх прежнего содержимого. Если в качестве входного и выходного файла указать один и тот же файл, данные могут исчезнуть до их чтения.

```
# Опасный пример. Не выполняйте его.
grep 'ERROR' incident.log > incident.log
```

`>>` добавляет данные в конец файла. При повторных запусках одинаковые строки будут накапливаться. Если каждый раз нужно заново создать файл с тем же содержимым, используют `>`.

Поток stderr обычно не попадает в файл при простом `>`. Оператор `2>` направляет stderr в отдельный файл.

5.6 Конвейер передаёт данные без промежуточного файла

```
grep 'ERROR' incident.log | tail -n 5
```

`|` соединяет stdout команды слева непосредственно со stdin команды справа. Результат можно отфильтровать без промежуточного файла на диске. Однако stderr левой команды обычно не передаётся через этот конвейер.

При стандартных настройках Bash код завершения всего конвейера равен коду последней команды. Поэтому ошибка слева может остаться незаметной, если команда справа завершилась успешно. В строгих автоматических сценариях рассматривают `set -o pipefail`.

5.7 Кавычки, переменные и подстановка команд

До запуска команды оболочка обрабатывает и разворачивает введённый текст.

```
name='incident.log'
printf '%s\n' "$name"
```

- Одинарные кавычки `'...'` сохраняют внутренний текст почти буквально; переменная `$HOME` не подставляется.
- Двойные кавычки `"..."` удерживают текст с пробелами как один аргумент, но позволяют подставлять переменные и результат `$(...)`.
- Без кавычек содержимое переменной может неожиданно разделиться по пробелам или развернуться как шаблон имён файлов. Обычно пишите `"$name"`.

Подстановка команды `$(command)` запускает команду и вставляет текст её stdout в данную позицию.

```
current_user="$(id -un)"
printf 'USER=%s\n' "$current_user"
```

5.8 `&&` продолжает работу только после успеха

```
cd /var/log && pwd
```

Правая команда после `&&` выполняется только при коде завершения `0` у левой. Это помогает не редактировать и не удалять файлы в неправильном месте после неудачного перехода в каталог. Но слишком длинная цепочка в одной строке затрудняет поиск этапа, на котором произошла ошибка.

Вопросы в конце главы

Вопрос 1. Кто открывает `errors.txt` в команде `grep ERROR incident.log > errors.txt` — `grep` или оболочка?

Вопрос 2. Чем отличаются `>` и `>>` и каковы последствия ошибки?

Ответы на вопросы главы

Вопрос 1

Ответ: оболочка. Она открывает `errors.txt`, соединяет файл со стандартным выводом `grep` и затем запускает `grep`.

Вопрос 2

Ответ: `>` перезаписывает файл, `>>` дописывает в конец. Ошибочный `>` уничтожает прежнее содержимое; повторный ошибочный `>>` создаёт дублирующиеся строки.

Источники

- Руководства `man grep`, `man tail`, `man bash` в Ubuntu 24.04 (версии проверяются в рабочей системе при подготовке).
- GNU Bash Reference Manual (получение текста через веб 2026-09-18 завершилось по тайм-ауту; первичная проверка проводится по локальному руководству).

Глава 6. Пользователи, группы и вход в систему

6.1 Виды учётных записей и их роли

В Linux есть учётные записи не только для людей, но и для процессов ОС и служб. Внутри система различает пользователей по числовому UID (идентификатору пользователя), а группы — по GID (идентификатору группы). Сначала сравним назначения и права учётных записей.

Тип	Основная роль	Интерактивный вход
Администратор root	Особый пользователь для управления всей системой; UID равен 0.	Учётная запись существует, но прямой вход root по паролю в Ubuntu обычно отключён. Разрешённые пользователи выполняют административные команды через sudo.
Обычный пользователь	Выполняет работу человека; примеры — ssm-user, jduops.	При подходящей настройке и способе аутентификации может войти интерактивно. Обычный пользователь не обязательно имеет право на sudo.
Пользователь службы	Запускает процесс, например веб-сервер, с необходимым минимумом прав; пример — jduweb.	Часто не нуждается в интерактивном входе. Обычно в качестве оболочки указывают /usr/sbin/nologin, но это не обязательное свойство любой учётной записи службы.

Это разделение по назначению, а не три жёстко зафиксированных внутри Linux класса. Реальные возможности зависят от настроек учётной записи, групп, прав каждого файла, правил sudo и SSH. Существование root не означает, что прямой вход под ним разрешён.

Имя вроде ssm-user удобно человеку. Сведения о владельце файла или каталога внутри системы хранятся как числовые UID/GID.

```
id
id jduops
getent passwd root
getent passwd jduops
getent passwd jduweb
getent group ops
```

Без аргумента id показывает UID, основную и дополнительные группы текущего пользователя. С именем пользователя команда проверяет зарегистрированные сведения об этой учётной записи. getent passwd USER выводит имя, UID, GID основной группы, домашний каталог, оболочку входа и другие поля. /usr/sbin/nologin в последнем поле означает, что обычная интерактивная оболочка недоступна. getent получает сведения не только из локального файла, но и из настроенных служб имён; в организации это могут быть LDAP или Active Directory.

6.2 Основная и дополнительные группы

У каждого пользователя есть одна основная группа (primary group). В Ubuntu при создании пользователя часто автоматически создают одноимённую группу и назначают её основной, но это не обязательное правило. При необходимости основную группу можно изменить, например через usermod -g.

Кроме основной, пользователь может входить в несколько дополнительных групп (supplementary groups). Для доступа команды к общему каталогу обычно лучше добавить её участников в отдельную дополнительную группу, чем менять основную группу каждого пользователя.

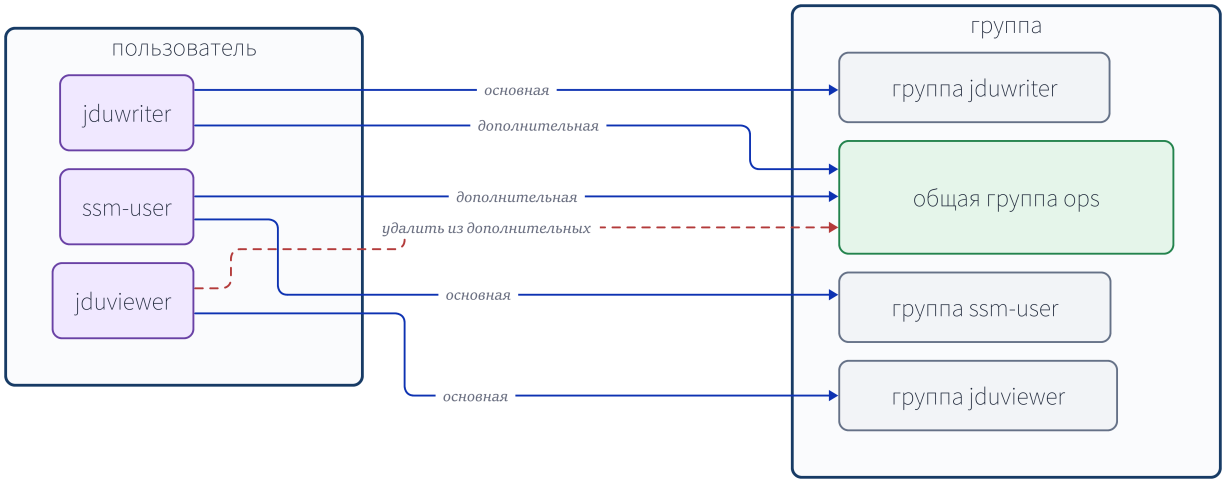


Рисунок 6-1. Связь основных групп пользователей с дополнительной группой для общих ресурсов.

Добавление и удаление членства в дополнительной группе выполняются командами:

```
sudo usermod -aG ops jduops
sudo gpasswd -d jduviewer ops
```

В `usermod -aG` параметр `-a` (append) сохраняет прежние дополнительные группы при добавлении новой. Если указать только `-G`, пользователь может лишиться членства во всех дополнительных группах, не перечисленных в команде. После изменения обязательно перепроверьте регистрацию через `id USER`.

Список в конце вывода `getent group ops` полезен для просмотра тех, у кого `ops` — дополнительная группа. Пользователь с `ops` в качестве основной группы может не отображаться в этом списке. Для надёжной проверки сочетайте вывод с `id USER`.

6.3 Запись об учётной записи и полномочия текущего процесса

Изменение членства в группе не обновляет автоматически данные уже работающей оболочки: запущенный процесс продолжает хранить полученный при старте набор групп. Новые сведения вступят в силу после нового входа или запуска нового сеанса.

```
id jduops
sudo su - jduops
id
```

Первый `id jduops` проверяет зарегистрированные сведения об учётной записи. `id` после `su -` показывает реальные UID/GID и группы нового процесса оболочки. При проверке доступа ядро ориентируется на учётные данные процесса, выполняющего действие, а не только на статическую запись в базе пользователей.

6.4 Открытие оболочки другого пользователя с помощью su

su позволяет выполнять команды с полномочиями другого пользователя. Здесь мы изучаем способ открыть его оболочку, чтобы продолжить работу от его имени.

```
su - jduops
id -un
pwd
exit
id -un
```

su - jduops запускает оболочку входа с полномочиями jduops. Знак - создаёт среду, близкую к обычному входу этого пользователя: переходит в его домашний каталог и инициализирует переменные среды, включая домашний каталог и оболочку. После переключения проверьте имя через id -un, а рабочий каталог через pwd.

Данные исходной учётной записи при этом не изменяются. Новая оболочка открывается внутри прежней. exit завершает её и возвращает на один уровень назад. Последняя команда id -un подтверждает возвращение к исходному пользователю.

При прямом запуске su - USER обычным пользователем обычно требуется пароль целевого пользователя. Пример выше предназначен для среды, где возможна такая аутентификация. В учебной Ubuntu мы используем sudo su - USER, как объясняется в следующем разделе. Прямой запуск su для пользователя без настроенного пароля может завершиться отказом.

su объясняется как substitute user. Английское substitute означает «заменять» или «подставлять вместо»: здесь заменяется пользователь, с полномочиями которого выполняются команды. Switch user, то есть «переключить пользователя», помогает запомнить действие, но служит его описанием.

6.5 Выполнение команды с полномочиями другого пользователя через sudo

sudo позволяет пользователю, которому это разрешено настройками, выполнять команды с полномочиями администратора root или другого пользователя. Обычная учётная запись сама по себе не даёт права на sudo. Настройки, например sudoers, определяют, кто, от чьего имени и какие команды может запускать. Административные полномочия следует ограничивать необходимыми операциями.

При обычной настройке без указания целевого пользователя команда выполняется от имени root. В примере только id -un с префиксом sudo работает от имени root. Следующий id -un выполняется в исходной оболочке и показывает имя исходного пользователя.

```
sudo id -un
id -un
```

Если обычная настройка требует пароль, sudo запрашивает пароль пользователя, который запускает команду. Это отличается от su, запрашивающего пароль целевого пользователя. Способ аутентификации зависит от настроек, в том числе возможна работа без запроса пароля.

Для выполнения одной команды от имени другого пользователя используется sudo -u USER -- COMMAND. Параметр -u USER задаёт целевого пользователя, а -- отмечает конец параметров sudo.

Такой вызов не открывает интерактивную оболочку. После завершения команды работа продолжается в исходной оболочке.

```
sudo -u jduweb -- test -r /srv/jdu-web/index.txt
echo $?
```

Это непосредственно проверяет чтение файла с полномочиями jduweb. test -r при успехе ничего не выводит. Сразу после него echo $? показывает 0, если чтение разрешено, и 1, если нет. Другая команда между ними изменит проверяемый код завершения. Чтение от имени root не доказывает, что доступ есть и у пользователя службы.

Для продолжения работы в оболочке другого пользователя на занятиях применяется следующая комбинация.

```
sudo su - jduops
id
pwd
exit
id -un
```

Это комбинация двух команд: sudo запускает su - jduops. При обычной настройке sudo проверяет разрешение и запускает su от имени root, после чего su открывает оболочку входа jduops. Вернитесь через exit и проверьте исходного пользователя командой id -un.

sudo объясняется как superuser do, то есть «выполнить от имени администратора». Вместе с substitute user для su это помогает запомнить назначение команд. При этом современный sudo позволяет через -u указать и пользователя, отличного от root. Названия похожи, но su и sudo являются разными командами. Различайте открытие оболочки другого пользователя для дальнейшей работы и выполнение разрешённой команды с полномочиями другого пользователя.

6.6 Отдельный пользователь ограничивает права службы

Пользователь службы помогает отделить права её процессов от прав людей и других служб. Если веб-сервер работает от имени jduweb, при проверке доступа используются UID/GID jduweb. Можно открыть ему чтение только нужных файлов.

```
getent passwd jduweb
id jduweb
```

/usr/sbin/nologin — программа, используемая вместо обычной оболочки для отказа в интерактивном входе. Учётная запись при этом существует. systemd может запускать процесс с правами пользователя, указанного в настройках службы. Невозможность человеку войти интерактивно не означает, что процесс службы с этим UID не может работать с файлами.

6.7 Учётная запись, аутентификация и проверка прав

Учётная запись (account) связывает имя пользователя с UID, основной группой, домашним каталогом, оболочкой входа и другими параметрами. Наличие на сервере учётной записи ssm-user означает, что сервер распознаёт это имя, но ещё не доказывает, что подключившийся человек вправе им пользоваться.

Аутентификация (authentication) подтверждает, что подключившийся действительно может использовать заявленную учётную запись. При входе по паролю проверяют секретный пароль, при аутентификации SSH по ключу — владение соответствующим закрытым ключом. Открытый ключ регистрируется, например, в `~/.ssh/authorized_keys` нужного пользователя на сервере. Закрытый ключ не отправляется на сервер для прямого сравнения.

Проверка прав доступа (authorization) отвечает на вопрос, разрешена ли этому субъекту определённая операция. В SSH настройки учётной записи и сервера могут ограничивать само подключение. После входа отдельно проверяются права читать или писать конкретные файлы и выполнять административные команды. Успешный вход не означает разрешение на любые действия.

Рассмотрим подключение SSH от имени `ssm-user` по трём понятиям.

1. Учётная запись: на сервере зарегистрирован `ssm-user`, и система может получить его UID и домашний каталог. Без регистрации обычный сеанс этого пользователя не открыть.
2. Аутентификация: клиент использует закрытый ключ, а сервер проверяет соответствие открытому ключу, зарегистрированному для `ssm-user`. При несоответствии вход отклоняется даже при существующей учётной записи.
3. Проверка прав: после прохождения ограничений SSH процесс сеанса работает с UID и группами `ssm-user`. Он не прочитает файл, доступный только другому пользователю. Право на `sudo` определяется отдельно.

Это схема для понимания понятий, а не точный порядок внутренних действий программы SSH. Правильный ключ сам по себе не гарантирует интерактивную оболочку: её могут запретить ограничения SSH или настройки учётной записи. И наоборот, `systemd` может запустить процесс службы с заданным UID без интерактивного входа по SSH. Существование учётной записи, разрешение войти и право процесса на операцию — разные решения.

Вопросы в конце главы

Вопрос 1. Чем основная группа отличается от дополнительной? Объясните на примере общего каталога.

Вопрос 2. Почему изменение членства в группе не действует в уже открытой оболочке? Что нужно запустить заново?

Вопрос 3. Почему `sudo -u jduweb -- test -r FILE` нельзя заменить чтением `cat FILE` от имени root?

Ответы на вопросы главы

Вопрос 1

Ответ: основная группа обязательно одна для каждого пользователя; в дополнительные группы он может входить по необходимости. Для общего каталога можно создать общую дополнительную группу и добавить в неё участников.

Вопрос 2

Ответ: работающая оболочка сохраняет UID и список групп, полученные при её запуске. Нужно снова войти или открыть новый сеанс оболочки с обновлёнными группами.

Вопрос 3

Ответ: чтение, разрешённое root, может быть запрещено процессу `jduweb`. `sudo -u jduweb` проверяет файл именно с полномочиями `jduweb`.

Источники

- Ubuntu Server documentation, User management (проверено 2026-09-24; root, обычные и системные пользователи).
- Ubuntu Server documentation, OpenSSH server (проверено 2026-09-24; аутентификация по открытому ключу и `authorized_keys`).
- Ubuntu 24.04 man pages, adduser(8), systemd.exec(5) (проверено 2026-09-24; пользователи служб и UID процесса).
- Linux man-pages, passwd(5), credentials(7) (проверено 2026-09-24; UID/GID, оболочка входа и учётные данные процесса).
- Локальные руководства Ubuntu 24.04: `man id`, `man getent`, `man usermod`, `man gpasswd`, `man su`, `man sudo` (проверяются при подготовке).
- util-linux: su(1) (проверено 2026-10-13: substitute user, оболочка входа и аутентификация)
- Ubuntu Server: Welcome to the terminal (проверено 2026-10-13: superuser do)
- Sudo: руководство sudo (проверено 2026-10-13: разрешения, целевой пользователь и аутентификация)

Глава 7. Права доступа и общий каталог

7.1 Права доступа определяют, кому что разрешено

Базовые права доступа Linux задают разрешения на чтение (r: read), запись (w: write) и выполнение (x: execute) для трёх категорий: владельца (u: owner user), группы-владельца (g: owner group) и остальных пользователей (o: others).

```
-rw-rw-r-- 1 root ops ... README.txt
drwxrwsr-x ... root ops ... /srv/jdu-share
```

Первый символ обозначает тип объекта: `-` — обычный файл, `d` — каталог. Следующие девять символов читают тройками для владельца, группы и остальных.

Ядро сопоставляет UID/GID процесса, который пытается выполнить действие, с метаданными объекта. Права трёх категорий не складываются. Если процесс работает от имени владельца, проверяется только тройка владельца. Иначе при принадлежности к группе объекта — только тройка группы. Если не подошло ни одно условие, действует тройка остальных.

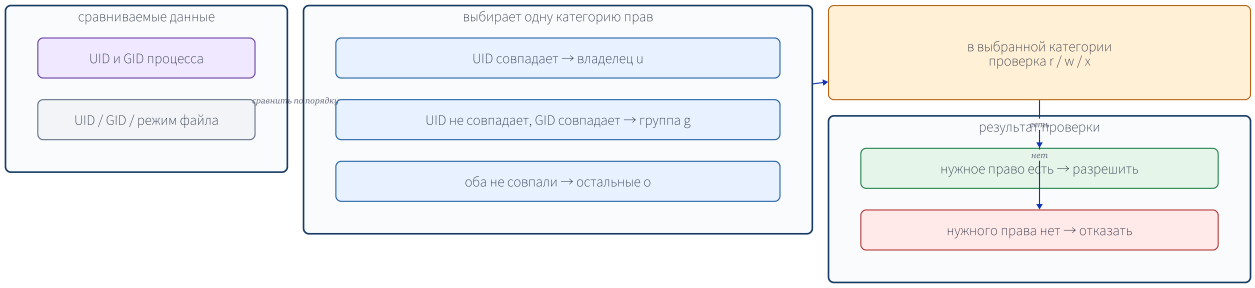


Рисунок 7-1. По учётным данным процесса выбирается категория u/g/o, затем её биты rwx определяют возможность действия.

7.2 Значение rwx у файла и каталога различается

Право	Обычный файл	Каталог
r	Читать содержимое.	Читать список имён внутри.
w	Изменять или дописывать содержимое.	Создавать и удалять имена внутри.
x	Выполнять как программу.	Проходить внутрь и достигать объекта по пути.

При праве записи в каталог иногда можно удалить расположенный в нём файл, даже если сам файл нельзя изменять: удаление меняет содержимое каталога. И наоборот, право чтения файла бесполезно, если нет права прохода `x` через один из родительских каталогов. `namei -l PATH` раскладывает путь на части и показывает права всех каталогов на нём.

7.3 Восьмеричная запись прав

В числовой записи права складываются как `r=4`, `w=2`, `x=1`.

- `6 = 4+2 = rw-`

- 7 = 4+2+1 = rwx
- 5 = 4+1 = r-x

Поэтому у обычного файла 664 означает rw-rw-r--, а у каталога 775 — rwxrwxr-x.

```
sudo chmod 664 /srv/jdu-share/README.txt
sudo chmod 2775 /srv/jdu-share
```

Режим 777 разрешает чтение, запись и выполнение даже категории «остальные». Он не подходит, если доступ должен быть только у участников общей группы.

7.4 chown и chmod

chown меняет владельца и группу-владельца, chmod — режим прав доступа.

```
sudo chown root:ops /srv/jdu-share
sudo chown root:ops /srv/jdu-share/README.txt
sudo chmod 2775 /srv/jdu-share
sudo chmod 664 /srv/jdu-share/README.txt
```

Изменение владельца или прав каталога не меняет автоматически уже существующие файлы и вложенные каталоги. Параметр -R позволяет менять всё дерево рекурсивно, но может затронуть неожиданные файлы.

```
stat -c '%U:%G %a %n' /srv/jdu-share /srv/jdu-share/README.txt
```

7.5 Какую проблему решает setgid у каталога

Обычно новый файл получает в качестве группы-владельца основную группу создавшего его процесса. Когда несколько пользователей работают в одном общем каталоге, файлы могут получить разные группы, и коллеги не смогут их редактировать.

Если у каталога включён бит setgid (Set Group ID), вновь создаваемые внутри файлы и каталоги наследуют группу-владельца родительского каталога вместо основной группы создателя. Например, при root:ops 2775 у /srv/jdu-share группа нового файла обычно будет ops.

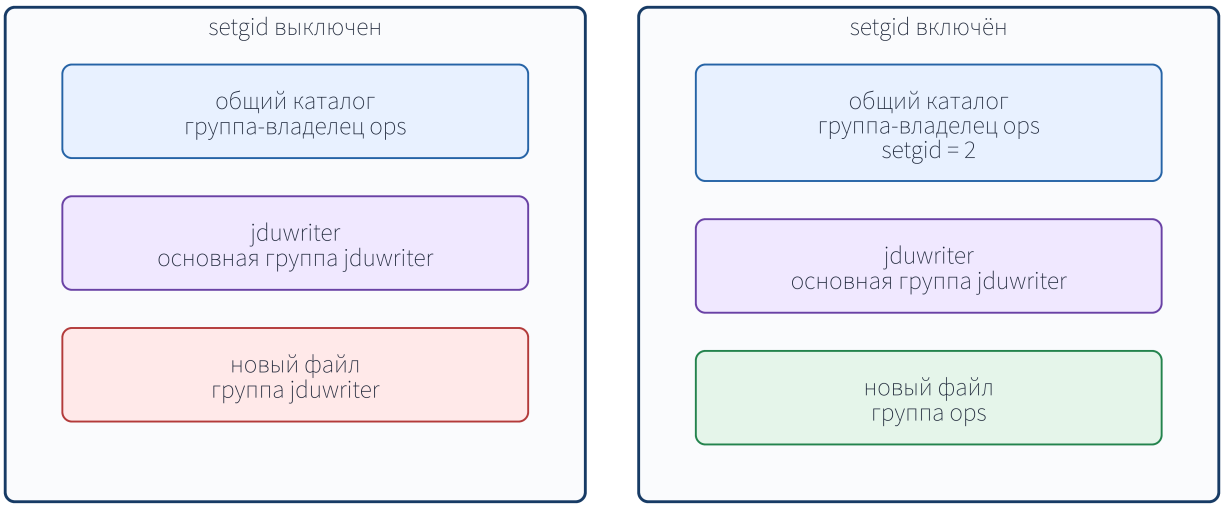


Рисунок 7-2. setgid согласует группу-владельца новых объектов с родительским каталогом; он не копирует владельца, содержимое или все биты прав.

Первая цифра `2` в восьмеричной записи режима обозначает setgid. В символьной записи на месте группового `x` показывается `s`.

```
drwxrwsr-x root ops /srv/jdu-share
```

Если setgid установлен без права прохода `x` для группы, отображается заглавная `S`. В общем каталоге группе обычно дают `rwx`, поэтому видно строчную `s`.

7.6 umask влияет на начальные права

setgid управляет наследованием группы, но не фиксирует все права нового файла как `664`. Фактический режим определяется исходными правами, которые запросила программа, с учётом битов, ограниченных umask (маской прав).

```
umask
touch /srv/jdu-share/sample.txt
stat -c '%U:%G %a %n' /srv/jdu-share/sample.txt
```

Например, при исходном режиме обычного файла `666` и umask `002` результатом будет `664` (`rw-rw-r--`). Но программа вправе запросить иной исходный режим. umask действует только в момент создания и не исправляет автоматически права уже существующего файла.

7.7 Проверяем разрешение и отказ от имени нужного пользователя

Недостаточно только посмотреть на метаданные — попробуйте действие с полномочиями интересующего пользователя.

```
sudo -u jduviewer -- test -r /srv/jdu-share/README.txt
echo $?
sudo -u jduviewer -- test -w /srv/jdu-share
echo $?
```

Код завершения `0` означает, что условие проверки истинно, `1` — ложно. Возможность создать файл внутри каталога зависит от прав `w` и `x` самого каталога. Проверка записи в существующий файл не заменяет проверку права создания в каталоге.

Если получен отказ, последовательно выясните пользователя и группы процесса, применённую категорию прав и недостающее право. Простое назначение `777` разрешит лишние действия и посторонним пользователям. Давайте доступ только тем, кому он нужен.

Вопросы в конце главы

Вопрос 1. Что означает каждая цифра режима 2775?

Вопрос 2. Что наследует новый файл в каталоге с setgid, а что не наследует?

Вопрос 3. Почему чтение файла от имени root не доказывает, что его прочитает пользователь службы?

Ответы на вопросы главы

Вопрос 1

Ответ: начальная `2` включает setgid. Затем `7` означает `rwx` для владельца, следующая `7` — `rwx` для группы, `5` — `r-x` для остальных. Для каталога символьная форма — `rwxrwsr-x`.

Вопрос 2

Ответ: новый файл обычно наследует группу-владельца родительского каталога. Владелец-пользователь и полный режим прав не копируются. Режим зависит от запроса создающей программы и umask.

Вопрос 3

Ответ: root может обходить некоторые обычные ограничения доступа. Для процесса службы проверка выполняется с его UID и группами и может дать отказ.

Источники

- Linux man-pages, path_resolution(7) (проверено 2026-09-18).
- Локальные руководства Ubuntu 24.04: `man chmod`, `man chown`, `man umask` (проверяются при подготовке).

Глава 8. Программа, процесс, PID и сигналы

8.1 Программа и процесс — не одно и то же

Программа — сохранённые на накопителе инструкции и связанные данные. Процесс — работающий экземпляр программы, загруженной в память и выполняемой под управлением ОС. Одну программу можно запустить несколько раз; каждый процесс получит собственный PID (идентификатор процесса).

Пакет — единица распространения и установки программы и её настроек. Служба — единица управления длительно работающей фоновой функцией. Не смешивайте эти отношения.

```
Установка пакета        → на диске появляются файлы программы.
Запуск программы        → создаётся процесс.
Управление юнитом systemd → можно запускать и отслеживать процесс службы.
```

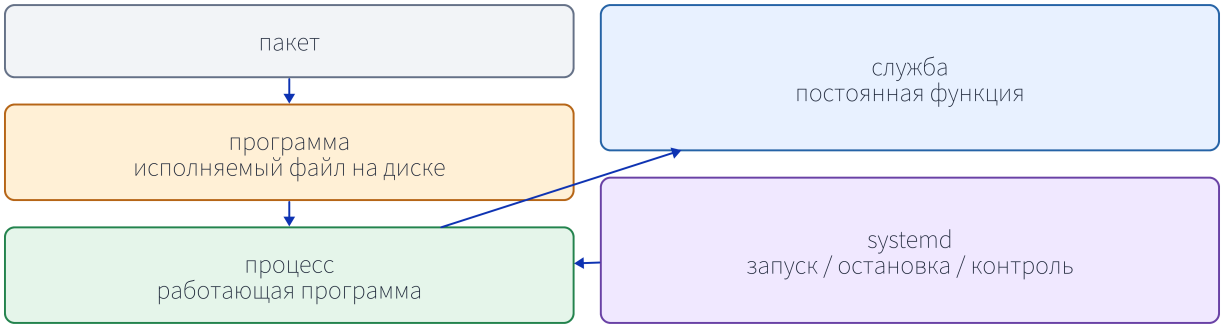


Рисунок 8-1. Различаем хранимую программу, работающий процесс, управляемую службу и распространяемый пакет.

8.2 PID и родительский процесс

Ядро назначает каждому запущенному процессу уникальное в данный момент положительное целое число — PID. Процесс может запустить дочерний процесс. По PPID (PID родителя) видно, какой процесс его создал.

```
ps -e -o pid,ppid,user,stat,comm,args
ps -p 1234 -o pid,ppid,user,stat,comm,args
```

`1234` здесь только пример: реальный PID меняется при запуске, а после завершения число может достаться другому процессу. Перед отправкой сигнала сверяйте не только PID, но и пользователя (`user`), имя команды (`comm`) и аргументы (`args`).

PID 1 принадлежит особому управляющему процессу, работающему с начала загрузки. В Ubuntu это обычно `systemd`.

8.3 Как найти процесс службы

В Ubuntu запуском служб и их состоянием управляет `systemd`. Настройка управляемого объекта называется юнитом (unit); имя юнита службы обычно заканчивается на `.service`. При запуске указанной в настройке программы появляется процесс службы. Юнит как объект управления и работающий сейчас процесс — не одно и то же.

Команда `systemctl` показывает состояние юнитов `systemd`. В примере перечисляются службы, затем проверяются состояние и главный процесс одной из них.

```
systemctl list-units --type=service
systemctl status systemd-journald.service --no-pager
```

Main PID в `status` — идентификатор главного процесса службы. Запишите показанный номер и подставьте его вместо `PID` в `ps -fp PID`. Так сведения об управляемой службе связываются с реальным процессом. `ps` показывает процессы, `systemctl` — управляемое состояние службы. Для остановки службы под управлением systemd используйте `systemctl stop ИМЯ_СЛУЖБЫ`, а не прямой сигнал её PID: остановкой будет управлять systemd. Автозапуск рассматривается в главе 10.

8.4 Работа на переднем и заднем плане

Когда команда запускается в терминале обычным способом, оболочка ждёт её завершения на переднем плане (foreground). Ввод с клавиатуры в это время в основном поступает этому процессу. `Ctrl+C` отправляет сигнал прерывания группе процессов переднего плана.

Символ `&` в конце команды запускает её в фоновом режиме (background), после чего оболочка принимает следующий ввод.

8.5 Сигнал — уведомление процессу

Сигнал (signal) — механизм асинхронного сообщения процессу о событии или управляющем запросе.

- TERM (SIGTERM) просит процесс завершиться штатно, оставляя возможность сохранить данные и освободить ресурсы. Обычно это первый сигнал при остановке.
- KILL (SIGKILL) заставляет ядро немедленно завершить процесс. Процесс не может перехватить сигнал и выполнить очистку. Это последняя мера, если обычный сигнал не помогает.
- INT (SIGINT) — сигнал прерывания, например от `Ctrl+C` в терминале.

```
kill -TERM PID
```

Несмотря на название, `kill` отправляет указанный сигнал процессу. В записи `pkill 1956` число `1956` воспринимается как шаблон имени процесса, а не как PID. Для конкретного PID используйте `kill`, для поиска по шаблону имени — `pkill`.

8.6 Остановка процесса не удаляет файлы

После завершения процесса сохранённые файлы программы, юнита и данных остаются на накопителе. И наоборот: удаление исполняемого файла не обязательно немедленно остановит уже загруженный в память процесс.

После завершения процесса `systemd` иногда перезапускает управляемую службу. Условия задаёт, например, `Restart=` в файле юнита. Поэтому «процесс завершился» и «служба останется остановленной» — разные утверждения.

8.7 Наблюдение через `/proc/PID`

```
printf 'PID=%s\n' "$$"
cat "/proc/$$/cmdline" | tr '\0' ' '
printf '\n'
readlink -f "/proc/$$/cwd"
```

`$$` — PID текущей оболочки Bash. Аргументы в `/proc/PID/cmdline` разделены нулевыми байтами (`\0`), поэтому `tr` заменяет их пробелами при выводе. `/proc/PID/cwd` — символическая ссылка на рабочий каталог процесса. Когда процесс завершается, его `/proc/PID` исчезает.

Так `/proc/PID` позволяет наблюдать не сохранённый файл программы, а сведения о работающем в данный момент процессе.

Вопросы в конце главы

Вопрос 1. Чем различаются пакет, программа, процесс и служба?

Вопрос 2. Почему перед отправкой сигнала нужно сверять PID через `ps`?

Вопрос 3. Если после TERM служба заработала снова, что проверить в файле юнита?

Ответы на вопросы главы

Вопрос 1

Ответ: пакет распространяет и устанавливает файлы программы и настроек. Программа — сохранённые инструкции. Процесс — работающий экземпляр программы. Служба — управляемая `systemd` или другой системой единица, предоставляющая длительную функцию.

Вопрос 2

Ответ: PID меняется между запусками и может повторно использоваться другим процессом. Нужно сверить также пользователя, команду и аргументы, чтобы не воздействовать на неправильный процесс.

Вопрос 3

Ответ: настройку перезапуска, например `Restart=`. После завершения процесса `systemd` может запустить его заново.

Источники

- Linux man-pages и Ubuntu 24.04: `man ps`, `man kill`, `man proc` (проверяются в рабочей системе).
- GNU Bash Reference Manual, Special Parameters (проверено 2026-09-24; `$$` — PID оболочки).
- Ubuntu 24.04 man page, systemd.service(5) (проверено 2026-09-24; `Restart=`).
- Ubuntu 24.04 man page, systemctl(1) (проверено 2026-09-24; `status` и `show`).

Глава 9. Пакеты и установка программ

9.1 Зачем нужны пакеты

Полезная программа не всегда состоит из одного исполняемого файла. Ей могут одновременно требоваться общие библиотеки, шаблоны настроек, руководства и файлы данных. Пакет (package) — единица распространения, объединяющая эти файлы, сведения о версии, зависимости и сценарии установки или удаления.

Ubuntu в основном использует формат пакетов Debian (`.deb`) и управляет ими через APT (Advanced Package Tool). APT получает сведения о доступных пакетах из репозиториев, разрешает зависимости и устанавливает нужные библиотеки. Запуск неизвестного сценария из интернета с правами root и установка проверенного пакета из доверенного официального репозитория — принципиально разные с точки зрения безопасности действия.

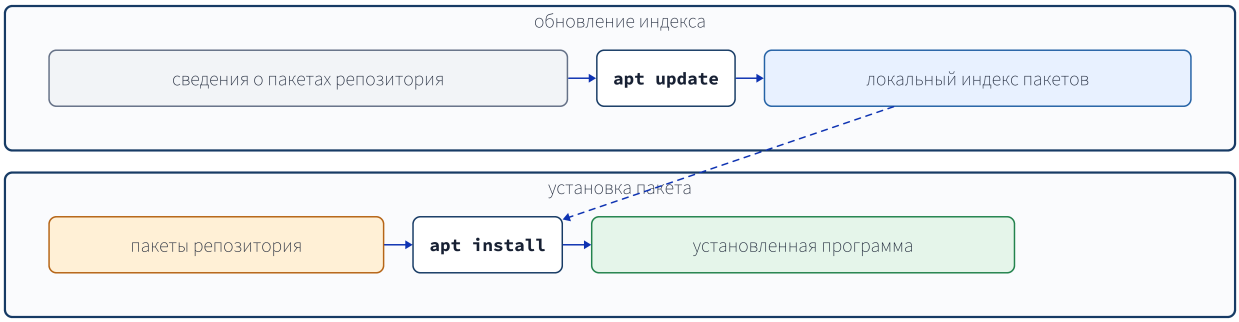


Рисунок 9-1. `apt update` обновляет индекс пакетов, а `apt install` загружает и устанавливает сам пакет.

9.2 Репозиторий и индекс пакетов

Репозиторий (repository) — сервер, предоставляющий пакеты и их метаданные, например версии и зависимости. Локальная система Ubuntu хранит индекс пакетов — сведения о том, какие версии доступны в подключённых репозиториях.

```
sudo apt update
apt show cmatrix
sudo apt upgrade
```

`apt update` обновляет из репозитория список доступных пакетов, не меняя сами уже установленные пакеты. `apt show cmatrix` позволяет заранее прочитать описание, версию и зависимости. `apt upgrade` на основе обновлённого списка обновляет установленные пакеты до новых версий. Перед продолжением изучите показанный список изменений. Это не переход на новый выпуск Ubuntu.

9.3 Установка и проверка запуска

```
sudo apt install cmatrix
command -v cmatrix
cmatrix
```

`sudo` требуется, поскольку меняются общесистемная база пакетов и защищённые каталоги, например `/usr/bin`. После установки `command -v` показывает, какой исполняемый файл оболочка

нашла для команды. Завершить `cmatrix` можно через `Ctrl+C`.

Установленный пакет и работающий сейчас процесс — разные состояния. После завершения команды пакет остаётся. Однако некоторые серверные пакеты настроены так, что при установке автоматически запускают службу. Фактическое состояние проверяйте через `systemctl`, а не предполагайте.

9.4 Проверка версии и принадлежности файла

```
dpkg-query -W -f='${Package} ${Version}\n' cmatrix
dpkg -S /usr/bin/apt
```

`dpkg-query` читает точную установленную версию из локальной базы пакетов. `dpkg -S PATH` выясняет, какой пакет предоставил указанный файл. `command -v` отвечает на вопрос «какой файл запустит оболочка?», а `dpkg -S` — «какому пакету принадлежит этот файл?». Путь `/usr/bin/apt` — пример. Если опубликованный путь команды является символьной ссылкой, сам этот путь может не принадлежать пакету. Тогда сначала определите путь к исходному файлу через `readlink -f` и проверьте его через `dpkg -S`.

9.5 Удаление и очистка

`apt remove` удаляет сам пакет. `apt purge` удаляет также его конфигурационные файлы. `apt autoremove` удаляет зависимости, которые больше не нужны другим пакетам. Это операции удаления, в отличие от `apt upgrade`; перед выполнением проверяйте, что именно будет удалено.

Вопросы в конце главы

Вопрос 1

Объясните различие между `apt update`, `apt upgrade` и `apt install`.

Вопрос 2

Чем отличаются вопросы, на которые отвечают `command -v cmatrix` и `dpkg -S /usr/bin/apt`?

Вопрос 3

Приведите случай, когда пакет установлен, но соответствующий процесс не работает.

Ответы на вопросы главы

Вопрос 1

Ответ: `apt update` обновляет индекс доступных пакетов, `apt upgrade` обновляет уже установленные пакеты, `apt install` устанавливает указанный пакет.

Вопрос 2

Ответ: `command -v cmatrix` находит путь к команде, которую запустит оболочка. `dpkg -S /usr/bin/apt` находит пакет, предоставивший этот файл. Если опубликованный путь — символьная ссылка, сначала определите исходный путь, затем найдите принадлежащий ему пакет.

Вопрос 3

Ответ: после установки `cmatrix` программу ещё не запускали либо завершили через `Ctrl+C`. Пакет есть, работающего процесса `cmatrix` нет.

Источники

- Ubuntu Server documentation, Install and manage packages (проверено 2026-09-24).
- Ubuntu Server documentation, Managing your software (проверено 2026-09-24).
- Локальные руководства Ubuntu 24.04: `man apt`, `man dpkg-query`, `man dpkg` (проверяются при подготовке).

Глава 10. systemd и службы

10.1 Что такое служба

Служба (service) — программа, которую система управляемо запускает в фоновом режиме для длительного предоставления функции без ручного запуска из терминала каждый раз. Примеры — веб-сервер, SSH-сервер, синхронизация времени и управление журналом. Не каждый процесс является службой, и не каждой службе нужен сетевой порт для ожидания соединений.

В Ubuntu 24.04 `systemd` запускается как процесс PID 1 и централизованно управляет многими службами. Важно: `systemd` не выполняет работу каждой службы сам. Это менеджер служб, отвечающий за запуск, остановку, зависимости, наблюдение за состоянием и автоматический запуск при загрузке.

10.2 Файл юнита — настройка, процесс — работа

`systemd` управляет ресурсами через общую единицу — юнит (unit). В файле юнита `.service` задаются правила: какой исполняемый файл запустить, от имени какого пользователя и в каком рабочем каталоге.

```
[Service]
User=jduapp
WorkingDirectory=/srv/jdu-status
ExecStart=/usr/bin/python3 /srv/jdu-status/server.py
```

- `User` — пользователь, от имени которого работает процесс службы.
- `WorkingDirectory` — текущий каталог процесса при запуске.
- `ExecStart` — абсолютный путь к программе и аргументы.

```
systemctl cat jdu-status.service
```

`systemctl cat` показывает файл юнита на диске и дополняющие настройки (drop-in). Если после изменения файлов не выполнить `daemon-reload`, показанный текст может не совпадать с конфигурацией, загруженной в `systemd`.

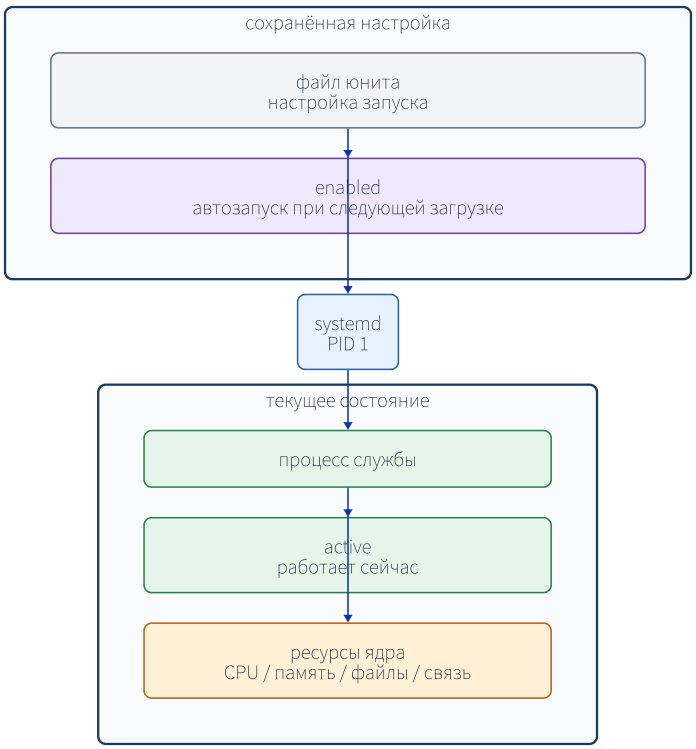


Рисунок 10-1. Юнит задаёт правила, systemd управляет, а процесс службы выполняет реальную работу.

10.3 active и enabled — разные состояния

```
systemctl is-active jdu-status.service
systemctl is-enabled jdu-status.service
```

- active / inactive — служба сейчас работает в памяти или остановлена.
- enabled / disabled — включён или выключен её автоматический запуск при загрузке системы.

Возможны оба сочетания: «active, но disabled» означает работающую сейчас службу без включённого автозапуска; «inactive, но enabled» — остановленную сейчас службу, которая должна запускаться при следующей загрузке. Немедленный `start` и настройка автозапуска `enable` — независимые операции.

```
sudo systemctl start jdu-status.service
sudo systemctl enable jdu-status.service
```

`enable --now` сочетает включение автозапуска с немедленным запуском.

10.4 start, stop, restart и reload

- start запускает остановленную службу.
- stop останавливает работающую службу.
- restart останавливает и снова запускает её; возникает краткий перерыв в доступности.
- reload просит уже работающий процесс перечитать его настройки без остановки. Не каждая служба это поддерживает.

`systemctl daemon-reload` перечитывает изменённые файлы юнитов самим systemd. Это не то же самое, что `reload` настроек работающего процесса службы.

10.5 Сверка настроек юнита с реальным процессом

```
systemctl status jdu-status.service
systemctl show --property MainPID --value jdu-status.service
```

В главе 8 мы находили процесс по Main PID. Теперь сопоставим `User=`, `ExecStart=`, `WorkingDirectory=` в юните с реальным процессом. `systemctl status` показывает загрузку и состояние юнита, Main PID и последние связанные записи журнала. Если вывод открылся в постраничном просмотрщике, нажмите `q`. `systemctl show` выдаёт значение конкретного свойства.

Получив Main PID, проверьте соответствие процесса настройкам. Число `1234` ниже служит только примером: замените его на показанный на вашем сервере Main PID.

```
ps -p 1234 -o pid,user,comm,args
sudo readlink -f /proc/1234/cwd
```

Настройка юнита	Где проверить у процесса
User=	Поле USER в `ps`.
ExecStart=	Поле args в `ps` и `/proc/PID/cmdline`.
WorkingDirectory=	Цель символической ссылки `/proc/PID/cwd`.

10.6 active не доказывает исправность функции

Процесс может числиться `active`, но не отвечать пользователю: нет прав чтения файла содержимого, сокет слушает не на том IP-адресе или приложение выдаёт исключение. В следующей главе мы расширим проверку от состояния службы к процессу, сокету, HTTP-ответу и журналу.

10.7 `systemctl status` не перечисляет все процессы

Без указания юнита `systemctl status` показывает сводку системы и дерево процессов; отдельные службы не обязательно удобно видны. Для проверки конкретной службы явно называйте её юнит.

```
systemctl status systemd-journald.service --no-pager
systemctl list-units --type=service 'systemd-*'
```

Существует много процессов, не являющихся непосредственно процессами юнитов служб, например запущенная человеком командная оболочка. Для общего списка процессов используйте `ps`, а для состояния служб под управлением systemd — `systemctl`.

Вопросы в конце главы

Вопрос 1

Различите роли файла юнита, systemd и процесса службы.

Вопрос 2

Может ли служба быть одновременно `active` и `disabled`? Что это значит?

Вопрос 3

Как по `/proc` проверить, применён ли `WorkingDirectory` к работающему процессу?

Ответы на вопросы главы

Вопрос 1

Ответ: файл юнита содержит правила запуска. systemd использует их для управления. Работающий процесс службы выполняет саму функцию.

Вопрос 2

Ответ: да. `active` говорит о текущей работе, `disabled` — что автозапуск юнита не включён. Так бывает после ручного `start`.

Вопрос 3

Ответ: узнайте Main PID через `systemctl show --property MainPID --value ИМЯ_ЮНИТА`, затем проверьте рабочий каталог процесса через `readlink -f /proc/PID/cwd` и сравните его с `WorkingDirectory=`.

Источники

- Ubuntu 24.04: systemctl(1), systemd.exec(5), proc_pid_cwd(5) (проверено 2026-09-24).
- Юниты и проверки P4/M4 открытого Lab v1.0.0 (первоисточник конкретных значений для Lab).

Глава 11. Наблюдаем службу через сокеты, HTTP и журнал

11.1 Как связаны служба, процесс и сокет

`systemd` запускает процесс службы по правилам юнита. Если программе нужна сеть, она через системные вызовы просит ядро создать сокет (socket). Ядро использует IP-адрес, протокол и номер порта, чтобы передать полученные по сети данные нужному процессу.

Точнее говорить не «служба сама создаёт сокет», а «программа, работающая как служба, использует предоставляемый ядром механизм сокетов». `systemd` управляет процессом и наблюдает за ним, но не выполняет вместо него весь обмен сетевыми данными.

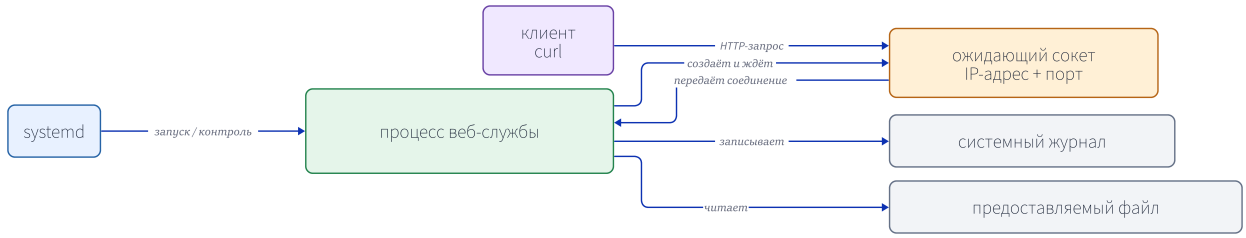


Рисунок 11-1. Управление (systemd), выполнение (процесс), сетевой механизм (ядро и сокет) и запрос клиента — разные этапы. У службы может быть несколько сокетов или вообще не быть сетевого обмена.

11.2 Протокол, IP-адрес и порт

TCP (Transmission Control Protocol) — один из протоколов транспортного уровня, часто используемых в веб-связи. Он подтверждает доставку и сохраняет порядок данных. Установка соединения и повторная передача подробно рассматриваются в последующем сетевом разделе.

IP-адрес обозначает сетевой интерфейс хоста. Номер порта помогает различить, какому коммуникационному входу внутри этого хоста предназначены данные. Порт сам по себе не программа. Ядро сопоставляет номер порта с ожидающим сокетом и передаёт данные соответствующему процессу.

- `127.0.0.1` — адрес обратной петли (loopback) IPv4. Напрямую извне он недоступен; обычно принимает соединения только внутри того же хоста.
- `0.0.0.0` — особая запись ожидания на всех локальных IPv4-адресах данного хоста, то есть на всех его сетевых интерфейсах.
- `[::]` — ожидание на всех локальных IPv6-адресах. В зависимости от настроек ОС может также принимать IPv4.

11.3 Читаем ожидающие соединения сокеты через `ss`

`ss` (Socket Statistics) показывает подробное состояние сокетов, которыми управляет ядро.

```
sudo ss -lntp | grep ':8081'
```

- `-l` — только сокеты в состоянии ожидания соединений (`LISTEN`).
- `-n` — числовые порты без обратного преобразования в имена служб.
- `-t` — только сокеты TCP.
- `-p` — сведения об использующем сокет процессе, включая имя и PID; иногда нужны административные права.

Схематический пример вывода:

```
State  Recv-Q Send-Q Local Address:Port Peer Address:Port Process
LISTEN 0      5      127.0.0.1:8081  0.0.0.0:*      users:(("python3",pid=2451,fd=3))
```

Читайте сначала состояние `LISTEN`, затем локальный адрес с портом `127.0.0.1:8081`, затем процесс и `pid=...`. Сверьте адрес и порт, а также PID с Main PID службы из `systemd`. Число `2451` дано только для примера: в своей системе читайте реально выведенный PID.

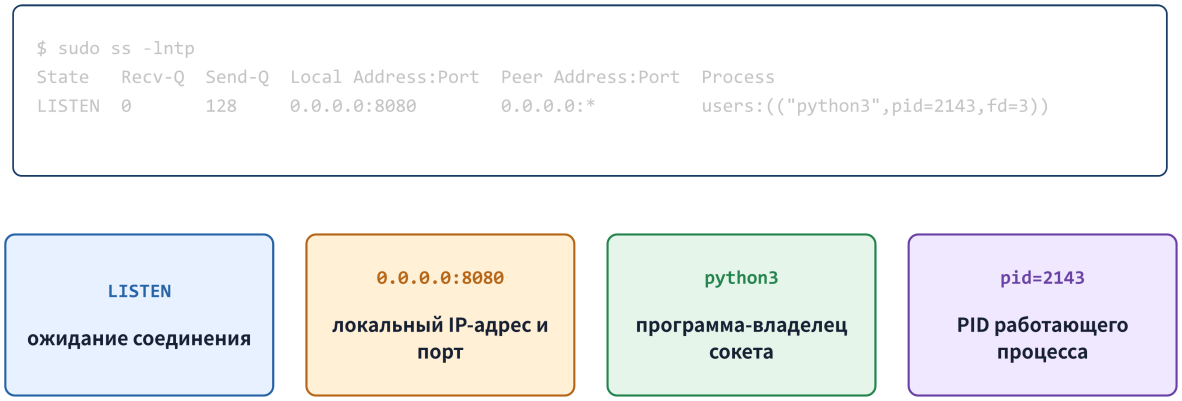


Рисунок 11-2. Строка `ss -lntp` разбирается на состояние, адрес, порт и данные процесса.

11.4 Клиент, сервер и HTTP

Процесс сервера создаёт сокет и ожидает соединения и запросы клиентов. Клиент подключается к сокету сервера и посылает запрос. В нашей практике команда `curl` на том же хосте Ubuntu выступает клиентом, а процесс `jdu-web.service` — сервером.

```
curl -i http://127.0.0.1:8081/
```

В URL `http://127.0.0.1:8081/` часть `http` обозначает протокол, `127.0.0.1` — адрес хоста, `8081` — порт, `/` — путь запроса. Параметр `-i` выводит не только тело ответа, но и заголовки HTTP.

```
HTTP/1.0 200 OK
Content-Type: text/plain

(тело ответа)
```

Код `200 OK` относится к уровню приложения HTTP. Успешное соединение TCP, получение заголовков HTTP и правильное содержание тела — три разных результата, которые нужно проверять отдельно.

11.5 Открытый и закрытый порт

```
curl --max-time 2 http://127.0.0.1:18081/
sudo ss -lnt | grep ':18081'
```

Если на `127.0.0.1:18081` никто не ожидает соединение, запрос обычно отклоняется и `curl` показывает `Connection refused`. Отсутствие этого адреса и порта в `ss -lnt` указывает на отсутствие ожидающего TCP-сокета. `--max-time 2` ограничивает всю операцию `curl` двумя секундами.

Наличие ожидающего сокета не гарантирует HTTP-ответ: после соединения серверная программа должна обработать запрос. Тайм-аут лишь означает, что операция не завершилась вовремя; по одному этому результату нельзя заключить, что сокета нет.

11.6 Права пользователя службы на чтение

Чтобы веб-сервер вернул содержимое файла, его пользователь должен иметь нужные права на сам файл и все родительские каталоги — чтение и проход.

```
id jduweb
stat -c '%U:%G %a %n' /srv/jdu-web/index.txt
namei -l /srv/jdu-web/index.txt
sudo -u jduweb -- test -r /srv/jdu-web/index.txt
echo $?
```

Невозможность интерактивно войти через `su -` от имени `jduweb` не мешает проверке. Для учётной записи службы интерактивная оболочка обычно отключена. Через `sudo -u` можно проверить доступ с её учётными данными, не открывая вход для человека.

11.7 Откуда берутся записи журнала

Во время работы программа может выдавать разные сообщения. В системе с `systemd` демон `systemd-journald` собирает stdout, stderr процессов служб и системные сообщения и хранит их в двоичном формате. Команда `journalctl` служит для чтения и поиска накопленных записей.

```
curl -i http://127.0.0.1:8081/
sudo journalctl -u jdu-web.service --no-pager -n 20
```

`-u` ограничивает вывод нужным юнитом, `-n 20` показывает двадцать последних записей, `--no-pager` выводит их без постраничного просмотрщика. Если сервер запрограммирован записывать обращения, появится строка для запроса. Наличие и текст записи зависят от реализации и настроек программы.

11.8 От настройки службы до ответа на запрос

На примере веб-службы настройки, выполнение, сеть, ответ и журнал связаны так:

```
Файл юнита → systemd → процесс службы
                ↓ использует сокеты ядра
          ожидающий сокет (IP-адрес и порт)
              ↓ подключение и запрос клиента
          обработка запроса процессом
            ├── HTTP-ответ
            └── при необходимости запись в журнал
```

Файл юнита определяет способ запуска, но запрос обрабатывает процесс службы. Ожидающий сокет — точка приёма соединений; HTTP-ответ — результат работы программы. Журнал помогает позднее понять ход обработки и ошибки. `systemctl` показывает состояние службы, `ss` — сокет, `curl` — HTTP-ответ, `journalctl` — собранные записи. Это сведения о разных этапах, а не четыре имени одного и того же факта.

Вопросы в конце главы

Вопрос 1

Назовите две возможные причины неправильного HTTP-ответа при состоянии службы `active`.

Вопрос 2

Чем отличаются `127.0.0.1:8081` и `0.0.0.0:8081` с точки зрения доступа извне?

Вопрос 3

Для чего сопоставлять PID процесса из `ss` с Main PID из systemd?

Ответы на вопросы главы

Вопрос 1

Ответ: например, программа слушает другой адрес или порт; либо её пользователь не может прочитать файл содержимого.

Вопрос 2

Ответ: `127.0.0.1:8081` принимает соединения внутри того же хоста. `0.0.0.0:8081` слушает на всех его локальных IPv4-адресах. Фактическая доступность извне зависит также от межсетевого экрана и сети.

Вопрос 3

Ответ: чтобы убедиться, что ожидающий сокет принадлежит процессу нужной управляемой службы. У многопроцессной службы PID сокета может отличаться от Main PID.

Источники

- Linux man-pages, socket(7) (проверено 2026-09-18).
- Linux man-pages, connect(2) (проверено 2026-09-24).
- curl, How to use curl (проверено 2026-09-24).
- Локальные руководства Ubuntu 24.04: `man ss`, `man curl`, `man journalctl` (проверяются при подготовке).
- Программы, юниты и проверки P5/M5 открытого Lab v1.0.0 (первоисточник значений конкретной среды).

Глава 12. Удалённая работа и передача файлов через SSH

12.1 Источник и адресат соединения

При управлении другим компьютером различайте место, где вводится команда, и место, где она исполняется. Сторону, начинающую SSH-соединение, называют локальной (local), адресата — удалённой (remote). Это названия с точки зрения текущего действия. Локальная сторона не обязательно физический ПК у вас на столе. Если SSH запускается из облачного терминала в браузере, источником соединения становится среда этого терминала.

```
Среда ввода команды (локальная)
        ↓ соединение SSH
Сервер выполнения (удалённый)
```

После подключения могут измениться имя пользователя, хоста и рабочий каталог. Выполните `id -un`, `hostname`, `pwd` до входа — получите сведения об источнике. Те же команды после входа показывают удалённую сторону. `exit` закрывает удалённую оболочку и возвращает к локальной. Не определяйте среду только по внешнему виду приглашения.

12.2 Что защищает SSH

SSH (Secure Shell) создаёт зашифрованный канал между компьютерами и позволяет использовать удалённую оболочку или команду. При соединении важны две проверки:

1. Проверка сервера (аутентификация хоста): действительно ли это нужный сервер? Для проверки используется его ключ хоста.
2. Проверка пользователя: разрешён ли заявленному пользователю вход? При аутентификации по открытому ключу используется пара ключей пользователя.

Затем команды, результаты и передаваемые файлы проходят по зашифрованному соединению. Но шифрование не создаёт само по себе сетевой маршрут до сервера. А успешный вход SSH не даёт права свободно читать и менять любые файлы: после входа действуют учётная запись и права удалённой ОС.



Рисунок 12-1. Ключ хоста подтверждает сервер, а ключ пользователя — право человека подключаться; роли этих ключей различаются.

12.3 Где находятся ключ пользователя и ключ хоста

При аутентификации по открытому ключу закрытый ключ (private key) пользователя хранится у SSH-клиента. Соответствующий открытый ключ (public key) обычно записывается на сервере в `~/.ssh/authorized_keys` пользователя, которому разрешён вход. Клиент доказывает владение закрытым ключом, не отправляя сам закрытый ключ серверу.

Ключ хоста (host key) определяет сам сервер. Сервер хранит закрытый ключ хоста, а клиент проверяет предъявленный сервером открытый ключ и обычно сохраняет доверенную запись в `~/.ssh/known_hosts`. При первом подключении сверяйте отпечаток ключа с данными из доверенного канала. Безусловное принятие незнакомого ключа может скрыть подмену сервера.

Тип ключа	Назначение	Где хранится закрытый ключ	Где проверяется открытый ключ
Ключ пользователя	Аутентификация подключающегося пользователя.	Источник соединения.	`authorized_keys` на сервере.
Ключ хоста	Проверка сервера.	Сервер.	`known_hosts` и другие сведения у клиента.

Не публикуйте закрытый ключ и не добавляйте его в Git. Если файл закрытого ключа доступен другим пользователям, SSH-клиент может отказаться его использовать. В обоих путях `~` означает домашний каталог пользователя соответствующей среды: на источнике и адресате это разные места.

12.4 Имя соединения и сетевой маршрут

SSH-клиент может хранить настройки адресата в локальном `~/.ssh/config`. Пример:

```
Host study-server
  HostName server.example.edu
  User learner
  IdentityFile ~/.ssh/id_ed25519
```

После этого `ssh study-server` использует указанные имя сервера, пользователя и файл закрытого ключа. Значение `Host` — псевдоним в конфигурации SSH, а не обязательно DNS-имя. Поэтому другой инструмент, не читающий конфигурацию SSH, может не разрешить такой псевдоним.

Если сервер недостижим напрямую, SSH может проходить через промежуточный узел или программу. В OpenSSH маршрут задают, например, `ProxyJump` и `ProxyCommand`. Маршрут отвечает на вопрос «как доставить соединение?», а аутентификация пользователя — «кому разрешён вход?». Даже правильный закрытый ключ бесполезен без сетевого пути.

12.5 Интерактивный вход и удалённая команда

`ssh study-server` после аутентификации открывает интерактивную оболочку адресата. Если после имени хоста указать команду, она выполнится на сервере и вернёт результат источнику.

```
ssh study-server 'id -un; hostname; pwd'
```

Здесь `id -un`, `hostname`, `pwd` выполняются на удалённом хосте. Внешние одинарные кавычки также не позволяют локальной оболочке заранее подставить `$HOME` или `$(...)`. Например, в `ssh study-server 'printf "%s\n" "$(hostname)" "$HOME"'` подстановку имени хоста и домашнего каталога выполняет удалённая оболочка. С внешними двойными кавычками значения подставила бы локальная оболочка.

Значение `~` тоже зависит от того, оболочка какого хоста и пользователя его интерпретирует. Локальный `~/notes.txt` и удалённый `~/notes.txt` — разные файлы.

```
ssh study-server 'printf "%s\n" "$HOME" > "$HOME/home-path.txt"'
ssh study-server 'printf "%s\n" "$HOME"' > local-result.txt
```

В первой строке и `$HOME`, и `>` обрабатываются на удалённой стороне, поэтому файл создаётся в домашнем каталоге адресата. Во второй строке `>` находится снаружи кавычек удалённой команды и обрабатывается локальной оболочкой. Полученный с сервера текст записывается в локальный `local-result.txt`. Один и тот же символ даёт разные результаты в зависимости от выполняющей его оболочки.

12.6 Передача файлов через SCP

`scp` переносит файлы с использованием SSH-соединения и аутентификации. Общая форма — `scp ИСТОЧНИК НАЗНАЧЕНИЕ`; удалённый путь записывают как `ХОСТ:ПУТЬ`.

```
scp notes.txt study-server:~/notes.txt
scp study-server:~/report.txt report.txt
```

Первая строка отправляет локальный `notes.txt` на сервер — загрузка (upload). Вторая получает `report.txt` с сервера — скачивание (download). `~` после двоеточия означает домашний каталог пользователя на удалённом хосте. Псевдоним из конфигурации SSH, например `study-server`, работает и для `scp`.

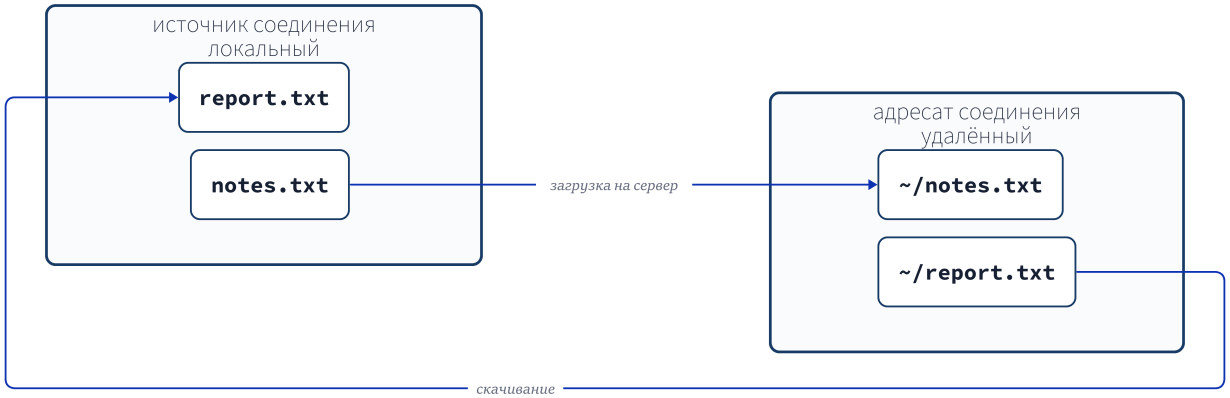


Рисунок 12-2. Направление отправки и скачивания определяется относительно стороны, выполняющей команду.

Если поменять местами источник и адресат, файл скопируется в обратном направлении. Следите и за возможной перезаписью одноимённого файла. Локальная и удалённая копии находятся в разных файловых системах.

12.7 Совпадение содержимого и свойства файлов

После переноса сравните вывод `sha256sum` для обеих копий:

```
sha256sum notes.txt
ssh study-server 'sha256sum ~/notes.txt'
```

Совпадение хешей — сильное подтверждение одинакового содержимого. Но хеш не включает владельца, группу, права доступа и местоположение. На сервере владельцем может стать другой пользователь, а режим доступа — отличаться. Свойства проверяют отдельно на адресате через `stat`.

```
ssh study-server 'stat -c "%U:%G %a %n" ~/notes.txt'
```

Это разные вопросы: `sha256sum` отвечает «одинаковы ли данные?», `stat` — «кому принадлежит файл на адресате и с какими правами он размещён?».

Вопросы в конце главы

Вопрос 1

Из облачного терминала в браузере устанавливается SSH-соединение с другим сервером. Какая среда локальная, а какая удалённая?

Вопрос 2

Для чего нужны пользовательский ключ и ключ хоста SSH? Где хранится закрытый ключ пользователя?

Вопрос 3

Чем SSH-псевдоним отличается от DNS-имени? Почему соединение может не установиться даже с правильным ключом?

Вопрос 4

На какой стороне в двух командах подставляется `$HOME` и сохраняется результат?

```
ssh study-server 'printf "%s\n" "$HOME" > "$HOME/home-path.txt"'
ssh study-server 'printf "%s\n" "$HOME"' > local-result.txt
```

Вопрос 5

Хеши SHA-256 двух файлов после `scp` совпали. Значит ли это, что совпадают владелец и права доступа?

Ответы на вопросы главы

Вопрос 1

Ответ: локальная среда — облачный терминал, где выполняется SSH, удалённая — сервер назначения. Физический ПК с браузером не обязательно является источником SSH.

Вопрос 2

Ответ: ключ пользователя подтверждает право пользователя на вход, ключ хоста помогает убедиться в подлинности сервера. Закрытый ключ пользователя хранится на стороне источника и не передаётся серверу.

Вопрос 3

Ответ: псевдоним задаётся в локальной конфигурации SSH и не обязательно известен DNS. Без сетевого маршрута к серверу соединения не будет даже при правильном ключе.

Вопрос 4

Ответ: в обеих строках `$HOME` раскрывается удалённой оболочкой. В первой `>` находится внутри кавычек и сохраняет файл на сервере. Во второй `>` вне кавычек и сохраняет результат у источника.

Вопрос 5

Ответ: нет. Хеш сравнивает содержимое, но не включает владельца и права. Их нужно отдельно проверить через `stat` на сервере.

Источники

- Ubuntu Server documentation, OpenSSH server (проверено 2026-09-24).
- OpenBSD manual pages, ssh(1), ssh_config(5), scp(1) (проверено 2026-09-24).
- GNU Coreutils, sha256sum(1) (проверено 2026-09-24).

Итоги книги

Программы на компьютере работают с помощью функций ОС. При работе с Linux полезно знать не только имя команды, но и понимать, к чему относится действие: к файлу, пользователю, процессу или сети. Тогда проще объяснить результат.

Наличие файла настроек, работающий процесс службы и ожидаемый сетевой ответ — разные состояния. Исправность одного не гарантирует остальные. Различайте владельца и права файла, пользователя службы, ожидающий сокет и записи журнала.

Дальше в курсе изучаются Nginx, BIND 9 и PostgreSQL. Их настройки различаются, но вопросы остаются общими: «От имени кого работает программа?», «Какие файлы ей нужны?», «Где она принимает соединения?», «Как проверить её работу?». Пусть изученные здесь основы ОС помогут разбираться в каждой новой системе.

Если нужно вспомнить написание и опции команд, обратитесь к [справочнику терминов и команд](../reference.md). Его разделы о чтении команды, `find`, `grep` и `tail` на коротких примерах объясняют `id -un` и способ разделять начальный путь поиска, условия и действие вывода.